

Scripting with Therefore™

www.therefore.net
© 2023 Therefore Corporation



All rights reserved. No parts of this work may be reproduced in any form or by any means - graphic, electronic, or mechanical, including photocopying, recording, taping, or information storage and retrieval systems - without the written permission of the publisher.

"JavaScript" is a trademark of Oracle Corporation in the United States.

Any other 3rd Party Products that are referred to in this document, are the property of, and may be either trademarks and/or registered trademarks of the respective owners in the USA and/or other countries. The publisher and the author make no claim to these trademarks.

While care has been taken in the preparation of this document, the publisher and the author assume no responsibility for errors or omissions, or for damages resulting from the use of information contained in this document.

VERSION: 2023 - 1

Table of Contents

Introduction

What is Scripting

SafeMode

Which Therefore applications use scripting?

Creating Scripts

1. Creating Common Scripts	10
1.1 Including Common Scripts	10
2. Creating New Scripts (Solution Designer).....	13
2.1 Creating New Scripts (External Application)	13

Objects

1. Objects	15
2. List of Objects.....	19

Advanced Scripts

1. Date and Time.....	34
1.1 Using ToDate	34
1.2 ToDateTime	34
2. Decimals	35
2.1 ToFloat	35
2.2 ToDecimal	35
3. Strings	37
3.1 Regexp	37
3.2 ReadPart	37
4. Content Connector and Document Loader.....	39
4.1 Setting up a multi-category indexing profile	39
4.2 FilesToSave	42
4.3 Using the XML Data Extractor	44
5. Capture client.....	51

5.1	Setting up a multi-category indexing profile (2)	51
6.	Indexing Profiles	55
6.1	GetField	55
6.2	SetField	59
6.3	SetCategory	66
7.	Workflow	70
7.1	Adding Rows	70
7.2	Updating Rows	71
7.3	Delete all Entries	72
7.4	Delete one Row	72
7.5	Document Creation Workflow Tasks	74
7.6	Building Loops with NextRequest	79
8.	Integration with MS Office	80
8.1	Extract table index field into an Office template	80
8.2	GetControl	80
8.3	GetFormField	85
8.4	GetCell	85
8.5	GetCells	87
8.6	GetProperty	87

Legal

Introduction

Therefore™ supports the use of the JScript engine from Microsoft to create scripts. JScript can run only in the presence of an interpreter, either in a Web server or a Web browser. JScript and JavaScript are the same scripting language, however, JScript supports conditional compilation. In this paper, we use the term JavaScript to refer to the scripting language.

This White Paper will provide you with a brief overview of scripting in general. It also provides you with a collection of objects, as well as frequently requested advanced scripts.

What is Scripting

What is scripting?

Within the context of Therefore™, scripting enables the automation of certain tasks – and this is implemented by writing specific scripts within the relevant scripting fields available in [some features](#).

What Scripting language do we use?

Therefore™ supports JScript (JavaScript), and this will be used in the examples displayed.



Therefore™ also supports VBScript. However, we recommend the use of JavaScript since it is more widely supported and easy to learn.

Why do we use scripting?

Scripting can enhance the functions of the Therefore™ system. By using a script, information can be added to the system.

E.g.,

- a script that enables your time and date settings to be set independently from that of your computer
- a script that updates a different category from the one in which the workflow runs
- a script to count how often a document has passed through a workflow
- etc.

Best Practice for scripting

It is recommended to use standard Therefore™ functions in order to keep Therefore™ as efficient as possible. Scripting should be used sparingly, and only to enhance functions. We recommend using out-of-the-box features to ensure the stability of your Therefore™ system.

Debugging

Please be aware that Therefore™ does not include a debugging function.

SafeMode

By default the scripting engine runs in safe mode where objects that are considered "unsafe" cannot be created. All of the following examples work with the SafeMode enabled. It is not recommended to disable the safe mode.



- Deactivating safe mode allows scripts full access to the computer.
- In Therefore™ Online systems, SafeMode is always ON and may not be deactivated.

Which Therefore applications use scripting?

You may encounter scripting in the following Therefore™ applications:

- Therefore™ Capture Client
- Therefore™ Document Loader
- Therefore™ Export Utility
- Therefore™ Solution Designer:
 - Indexing Profile
 - eForms
 - Common Scripts
 - Template Profiles
 - Workflow:
 - Automatic Tasks:
 - Update Index Data,
 - Embed Index Data,
 - Add File to Document,
 - Create New Document,
 - Call REST Function,
 - Change Category



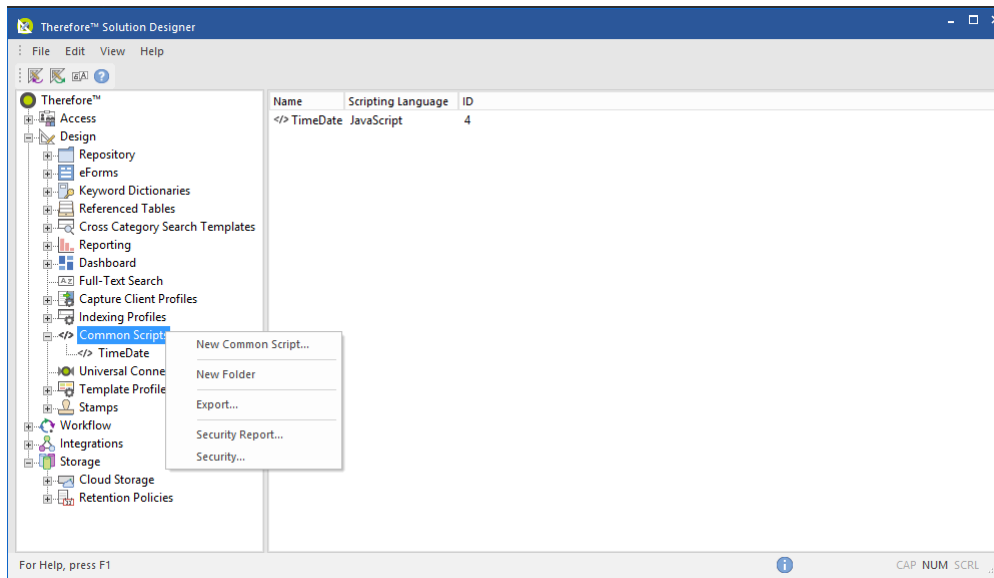
In this context, eForms refers to the configuration in the Therefore™ Solution Designer.

Creating Scripts

Find out how to create common scripts and new scripts.

1. Creating Common Scripts

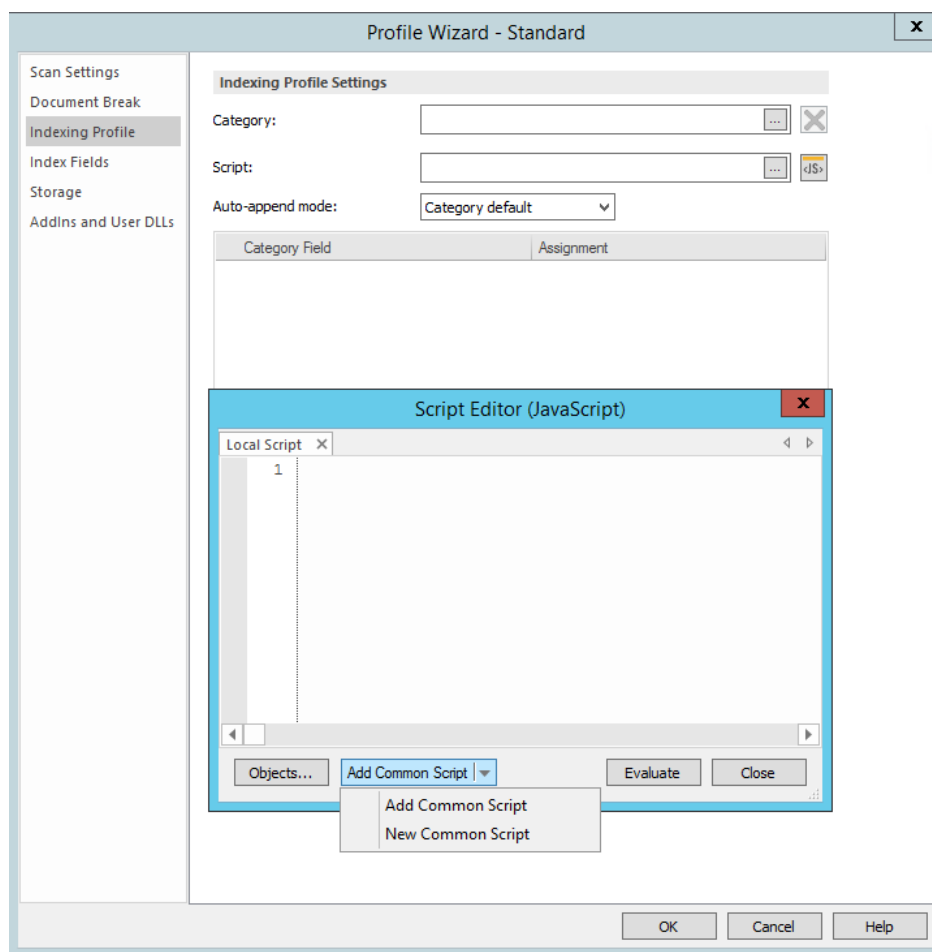
Access 'Common Script' from any Therefore™ application that has scripting enabled. This will allow you to use a script that has already been configured in the Therefore™ Solution Designer. Right-click on the 'Common Scripts' node and select 'New Common Script'.



1.1 Including Common Scripts

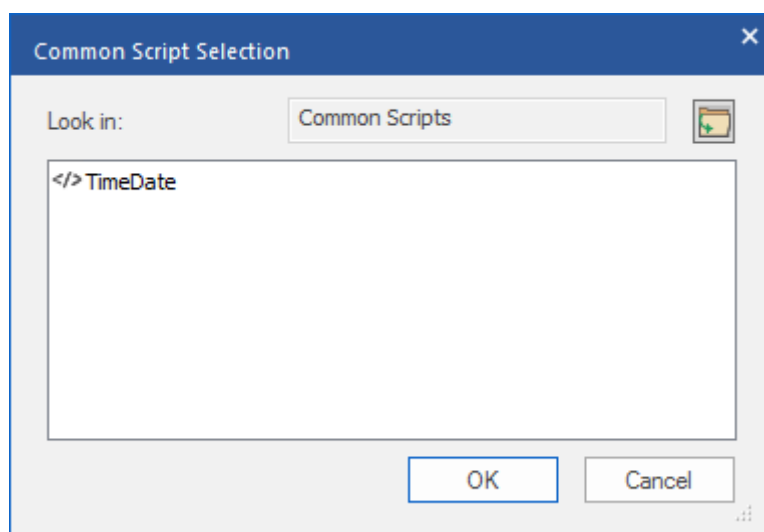
Including Common Scripts from other Therefore™ applications

Within various features and functionalities of Therefore™, the opportunities to including scripting is available. When a 'Script' field is displayed within the dialog, select either 'Add Common Script' or 'New Common Script'.



Add Common Scripts

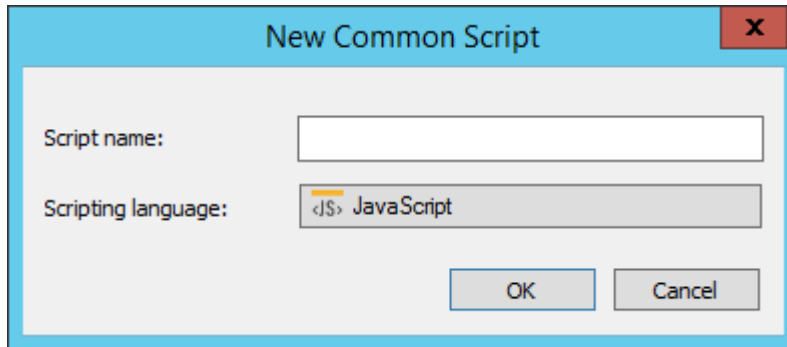
When clicking on the 'Add Common Script' option, a selection dialog will appear with pre-configured common scripts to choose from. Select a common script and click OK.



2. Creating New Scripts (Solution Designer)

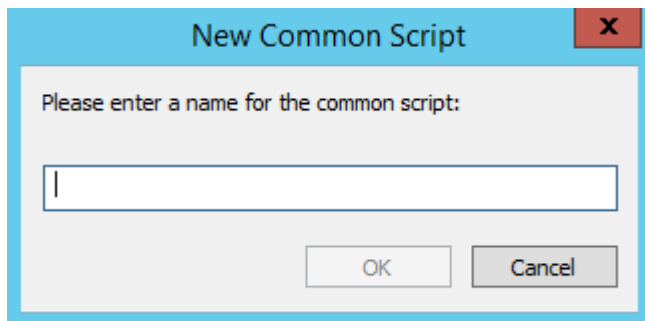
Creating New Scripts (within the Therefore™ Solution Designer)

Any “New Script” creates a new common script. A new script may be created in any application as long as it is evaluated. In the Therefore™ Solution Designer you can add as many new common scripts as you want.

A dialog box titled "New Common Script" with a red close button (X) in the top right corner. It contains two input fields: "Script name:" with an empty text box, and "Scripting language:" with a dropdown menu showing "<JS> JavaScript". At the bottom right are "OK" and "Cancel" buttons.

2.1 Creating New Scripts (External Application)

Within an external Therefore™ application (i.e. an application other than the Therefore™ Solution Designer), clicking on the 'New Common Script' option will prompt a dialog where the user names the new script.

A dialog box titled "New Common Script" with a red close button (X) in the top right corner. It contains a text prompt "Please enter a name for the common script:" above a single-line text input field. At the bottom right are "OK" and "Cancel" buttons.

Once a new name has been created, the user can then create a new script and then click 'Evaluate' in order to run it.

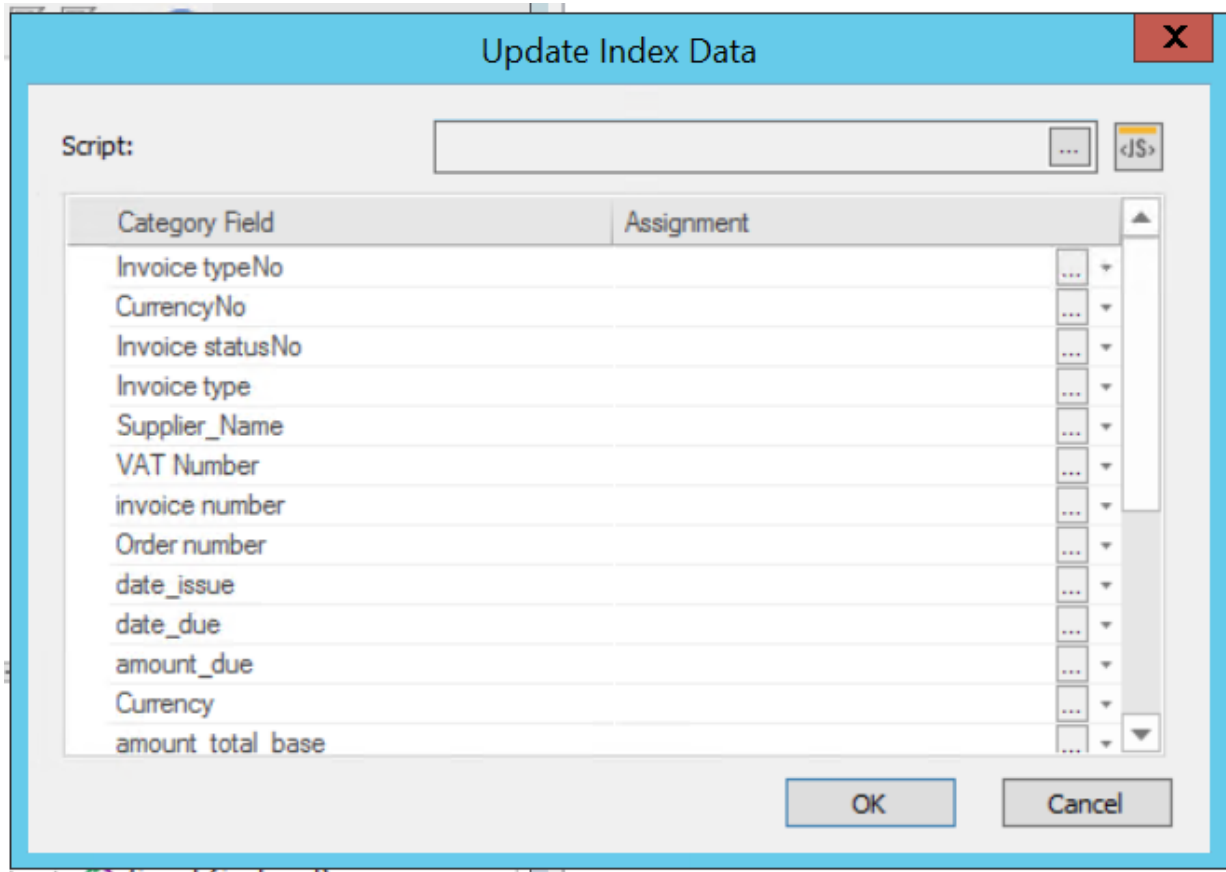
Objects

An object can be used in order to run an action on specific assignments. This is executed before an automated/configured task in Therefore™ has taken place. Objects can be used for simple tasks to take place alongside the main assignment.

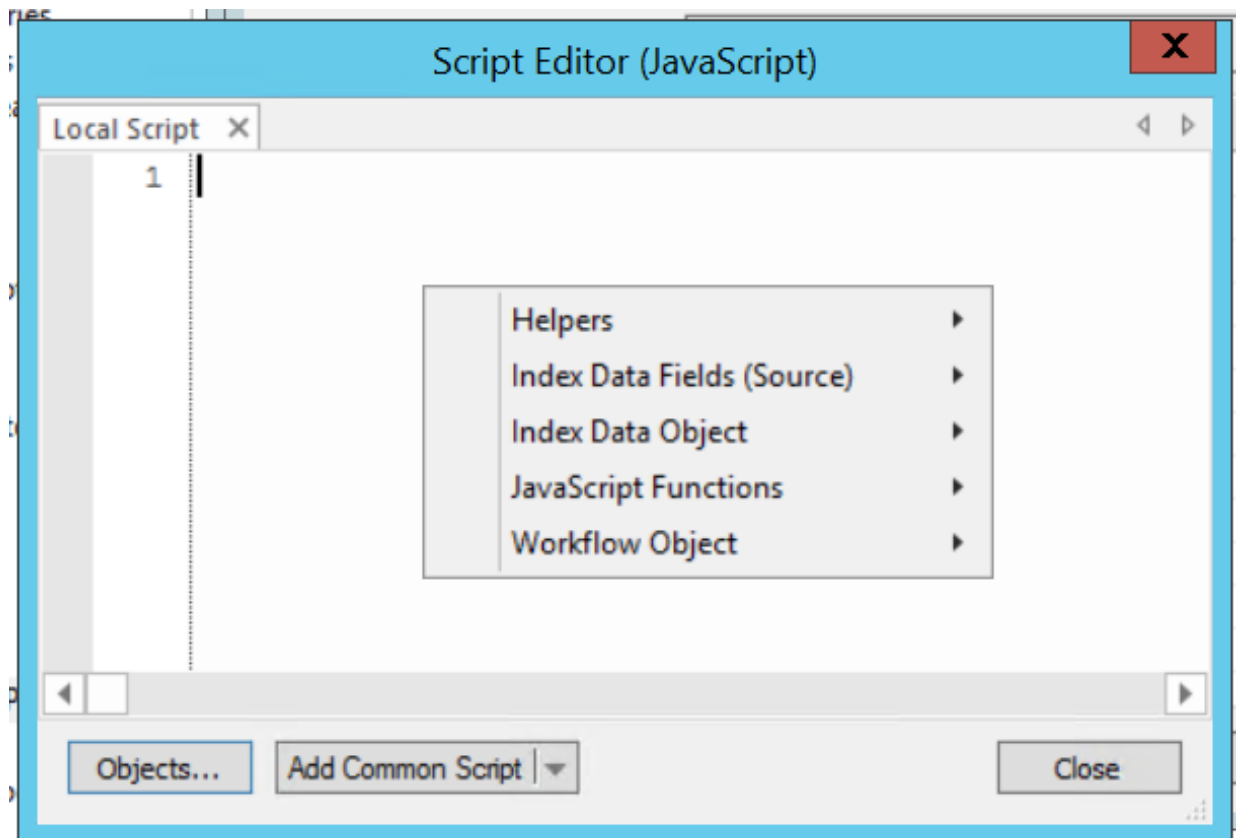
1. Objects

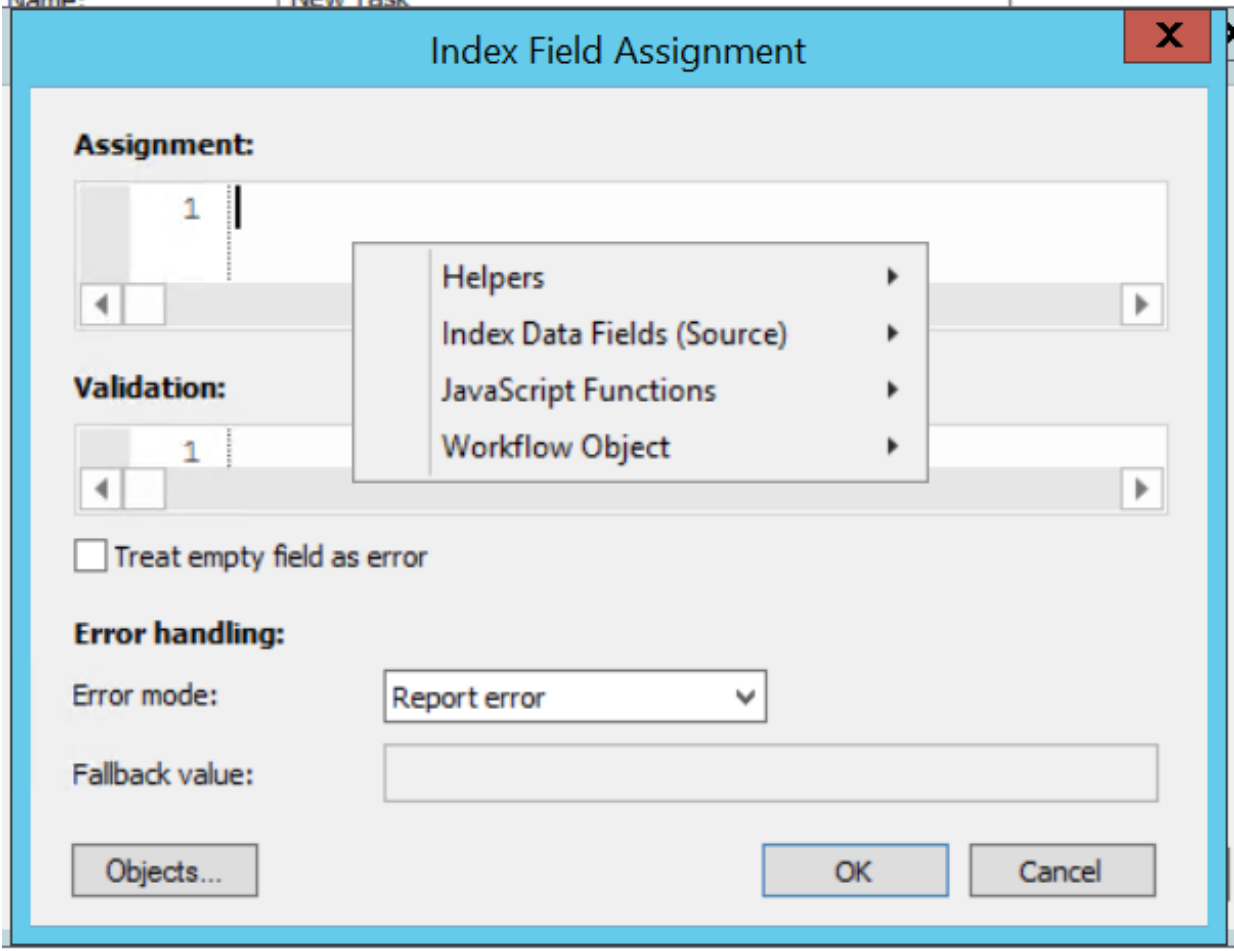
Basic tasks can be used to take actions in automatic tasks and field assignments within a workflow, e.g., in the 'Update Index Data' task as shown below.

A script can be written for both the general task (using the 'Script' field) and for individual field assignments using the '...' symbol.



These scripts can also contain functions and variables that can be called from the assignments. Right-click to open a list of available objections, scriptlets and functions.

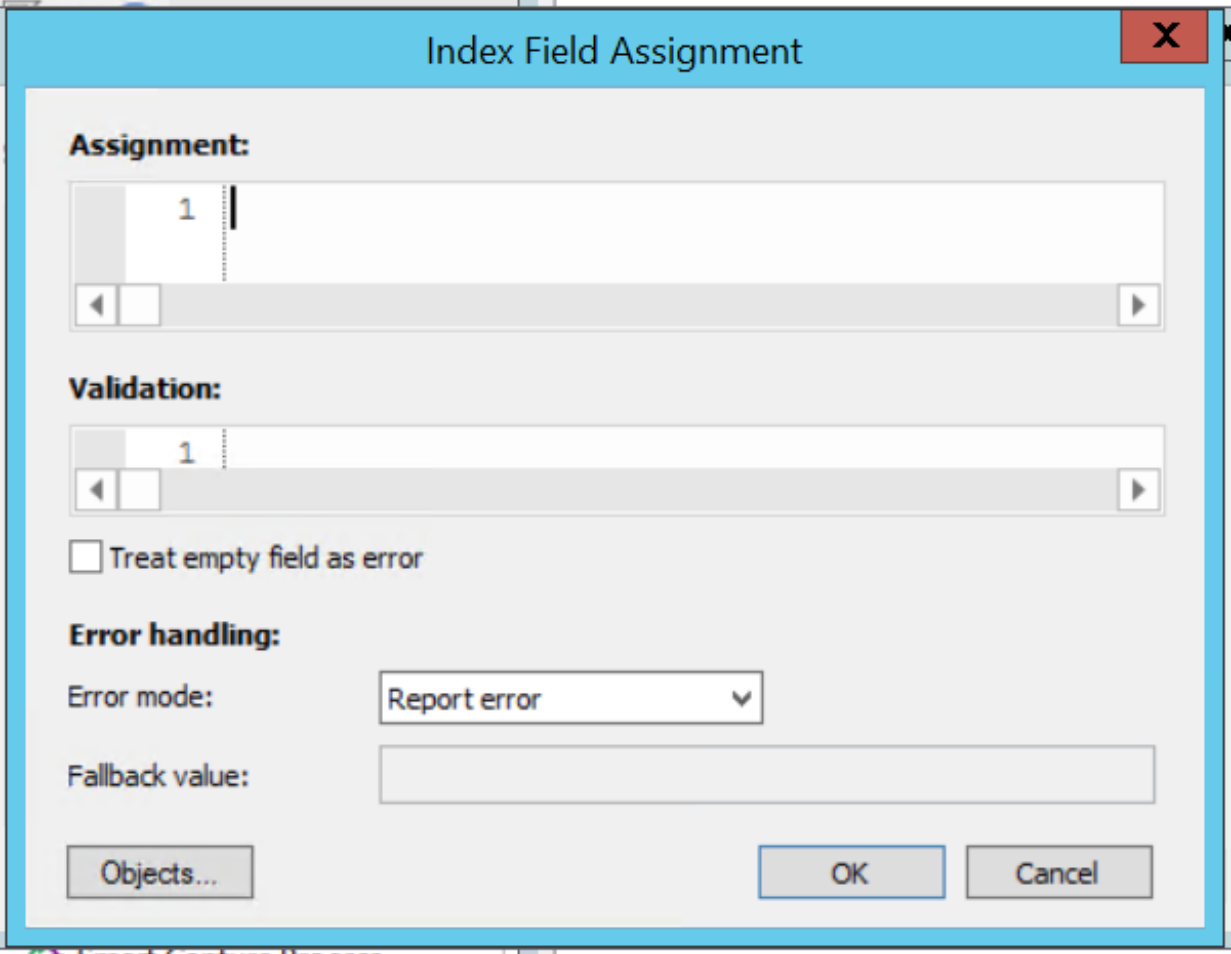




The image shows a software dialog box titled "Index Field Assignment". It has a blue header bar with a close button (X) in the top right corner. The dialog is divided into several sections:

- Assignment:** Contains a list box with the number "1" and a button with a left arrow. To its right is a menu with four items: "Helpers", "Index Data Fields (Source)", "JavaScript Functions", and "Workflow Object", each with a right arrow. Further right is another button with a right arrow.
- Validation:** Contains a list box with the number "1" and a button with a left arrow. To its right is a button with a right arrow.
- Error handling:** Includes a checkbox labeled "Treat empty field as error". Below it, "Error mode:" is followed by a dropdown menu currently showing "Report error". Below that, "Fallback value:" is followed by an empty text input field.
- Buttons:** At the bottom, there is an "Objects..." button on the left, and "OK" and "Cancel" buttons on the right.

Click on 'Object' to open a window with some predefined objects.



The image shows a software dialog box titled "Index Field Assignment". It has a blue title bar with a red close button (X) in the top right corner. The dialog is divided into three main sections: "Assignment:", "Validation:", and "Error handling:". Each section has a text input field with a small icon on the left and a scrollbar on the right. The "Assignment:" field contains the number "1". The "Validation:" field also contains the number "1". Below the "Validation:" field is a checkbox labeled "Treat empty field as error", which is currently unchecked. The "Error handling:" section contains two labels: "Error mode:" and "Fallback value:". The "Error mode:" label is followed by a dropdown menu showing "Report error". The "Fallback value:" label is followed by an empty text input field. At the bottom of the dialog, there are three buttons: "Objects..." on the left, and "OK" and "Cancel" on the right.

Index Field Assignment

Assignment:

1

Validation:

1

☐ Treat empty field as error

Error handling:

Error mode: Report error

Fallback value:

Objects... OK Cancel

2. List of Objects

Find a list of objects for workflow tasks below.

File Profile

Type of Object	Name	Description	Used with
File Object	File.Created	The creation time of the file.	Script & Assignments
	File.Exists	Returns True if the file exists, otherwise false.	Script & Assignments
	File.Extension	The extension of the file.	Script & Assignments
	File.FullPath	Gets the full path to the processed file.	Script & Assignments
	File.GetPath	Gets the folder name from file ascending. For example, GetPath(0) would return Folder2 for "C:/Folder1/Folder2/File.txt".	Script & Assignments
	File.GetPathFromRoot	Gets the folder name from the root descending. For example, GetPathFromRoot(0) would return Folder1 for "C:/Folder1/Folder2/File.txt".	Script & Assignments
	File.Modified	The last time the file was modified.	Script & Assignments
	File.Name	The full name of the file. To extract part of a file name, e.g. the characters 2 to 6 use Mid(File.Name,2,5).	Script & Assignments
	File.Size	The size of the file in bytes.	Script & Assignments
	File.Title	The title of the file. (e.g: test.docx - > test).	Script & Assignments
Index Data Object	ExecuteProfile	This function executes a File indexing profile given the indexing profile id and the filepath to the file. Usually used with an e-mail indexing profile, where we can get index data from an attachment too. (e.g PDF attachment).	Script
	GetField	Gets the current index data value of the specified field. See GetField for more information.	Script

	SetCategory	Gets the category for the current document. You can use an integer (category ID) or a string (category name). See SetCategory for more information.	Script
	SetField	Sets the value of the index data field. See SetField for more information.	Script
	SetCaseNo	Sets the CaseNo for this Index Data object.	Script
	SetCaseDef	Initializes this index data object, with a case definition. DO NOT USE THIS DIRECTLY ON THE PREDEFINED IndexData object (the predefined object is used when you don't write an object name in front of the function call). This should be used in conjunction with CreateThereforeObject. See Creating a New Case.	Script
	CreateCase	Use this after you have initialized the index data object with a case definition and after all mandatory fields are filled in. DO NOT USE THIS DIRECTLY ON THE PREDEFINED IndexData object (the predefined object is used when you don't write an object name in front of the function call). This should be used in conjunction with CreateThereforeObject. See Creating a New Case	Script
Profile Object	AddNewKeyword	With this function you can add a new keyword to an already existing keyword dictionary.	Script
	FilesToSave	Sets the files that should be saved with this document. See FilesToSave for more information.	Script
	NotApplicable	Call this function if you want to abort the indexing profile execution.	Script
	ReadPart	Reads a part of the text, specified with a mask. For example to extract 1234 from PO:1234 use: ReadPart (Extract("ExtractedStringName"), "PO*:*"). The "*" represents 'any text' and the "?" represents the section to be extracted.	Script & Assignments

	ScriptError	Stops the execution of the script and throws an error with the specified text.	Script
	ToDate	Converts a string in the specified format to a date. This is needed, for example, when the date on imported documents is in a different format to the system. Use "D", "M", "Y" for day, month, year respectively. Delimiters are interpreted strictly, for example "D-M-Y" the delimiter MUST be "-" in this case. To represent any delimiter use "~".	Script & Assignments
	ToFloat	Converts a string in the specified format to a float number. This is needed, for example, when the money format on imported documents is in a different format to the system.	Script & Assignments
	CreateThereforeObject	This function will create a new Therefore object. As parameter, an array is expected. The first element in the array should always be a string with the object name you want to create. Currently this can be used to create an Index data object. See Creating a New Case. Note: If you use the scripting help dialog, it will insert a predefined CreateThereforeObject call.	Script
Data Extractor Object (PDF)	Extract	Gets the value of the defined item name.	Script & Assignments
	ExtractList	Gets the value of the defined item name on all pages of the PDF (only applicable if the item name was defined with Page set to "All pages").	Script & Assignments
	GetFormField	Gets the value of the PDF form field	Script & Assignments
	SetErrorOnMissingTag	If set to True, the extract functions will throw an error if the item name is not found. Otherwise the Extract function will return empty.	Script
PDF Document Properties	GetAuthor	Gets the Author	Script & Assignments
	GetCreationDate	Gets the creation date	Script & Assignments

	GetCreator	Generally gets the application that created the PDF	Script & Assignments
	GetCustomProperty	Gets a custom property from the PDF. e.g. <code>GetCustomProperty("MyProp")</code>	Script & Assignments
	GetKeywords	Gets the keywords associated with the PDF	Script & Assignments
	GetModificationDate	Gets the last modification date	Script & Assignments
	GetProducer	Gets the PDF Producer	Script & Assignments
	GetSubject	Gets the PDF Subject	Script & Assignments
	GetTitle	Gets the PDF Title	Script & Assignments
Data Extractor Object (XML)	Extract	Gets the value of the first tag found. (e.g. <code>Extract("ExampleTag")</code> will extract the value of the first tag with name "ExampleTag").	Script & Assignments
	ExtractAttribute	Gets the value of the specified attribute of the specified tag. (e.g. <code>ExtractAttribute("ExampleTag", "att1")</code> will extract the attribute: att1 from the first tag with name "ExampleTag").	Script & Assignments
	ExtractList	Gets the value of each tag. (e.g. <code>ExtractList("ExampleTag")</code> will extract the values from all tags with name "ExampleTag").	Script & Assignments
	ExtractListAttribute	Gets the value of each attribute in the specified tag. (e.g. <code>ExtractListAttribute("ExampleTag", "att1")</code> will extract the attribute: att1 from all the tags with name "ExampleTag").	Script & Assignments
	ExtractListTable	Similar to <code>ExtractList</code> but ensures that any empty rows in a table are retained. Hence the table tag must be specified. (e.g. <code>ExtractListAttribute("TagName", "TableTag")</code>)	Script & Assignments
	ExtractListXPath	Can be used to execute an XPath query on the XML document.	Script & Assignments
	Count	Gets the count of nodes from the XPath query.	Script & Assignments

	SetErrorOnMissingTag	If set to true, every Extract function will throw an error if the specified tag was not found. Otherwise the Extract function will return "empty". Default is set to false.	Script
Data Extractor Object (Text-Line/Text-Page)	Extract	Gets the value of the defined item name.	Script & Assignments
	ExtractList	Gets the value of the defined item name on all pages of the text file (only applicable if the item name was defined with Page set to "All pages").	Script & Assignments

Profiles for Microsoft Office

Type of Object	Name	Description	Used with
Index Data Object	ExecuteProfile	This function executes a File indexing profile given the indexing profile id and the filepath to the file. Usually used with an e-mail indexing profile, where we can get index data from an attachment too. (e.g., PDF attachment).	Script
	GetField	Gets the current index data value of the specified field.	Script
	SetCategory	Gets the category for the current document. You can use an integer (category ID) or a string (category name).	Script
	SetField	Sets the value of the index data field.	Script
	SetCaseNo	Sets the CaseNo for this Index Data object.	Script
	SetCaseDef	Initializes this index data object, with a case definition. DO NOT USE THIS DIRECTLY ON THE PREDEFINED IndexData object (the predefined object is used when you don't write an object name in front of the function call). This should be used in conjunction with CreateThereforeObject. See Creating a New Case.	Script

	CreateCase	Use this after you have initialized the index data object with a case definition and after all mandatory fields are filled in. DO NOT USE THIS DIRECTLY ON THE PREDEFINED IndexData object (the predefined object is used when you don't write an object name in front of the function call). This should be used in conjunction with CreateThereforeObject. See Creating a New Case.	Script
Profile Object	AddNewKeyword	With this function you can add a new keyword to an already existing keyword dictionary.	Script
	FilesToSave	Sets the files that should be saved with this document. See FilesToSave for more information.	Script
	NotApplicable	Call this function if you want to abort the indexing profile execution.	Script
	ReadPart	Reads a part of the text, specified with a mask. For example to extract 1234 from PO:1234 use: ReadPart (Extract("ExtractedStringName"), "PO*:?"). The "*" represents 'any text' and the "?" represents the section to be extracted.	Script & Assignments
	ScriptError	Stops the execution of the script and throws an error with the specified text.	Script
	ToDate	Converts a string in the specified format to a date. This is needed, for example, when the date on imported documents is in a different format to the system. Use "D", "M", "Y" for day, month, year respectively. Delimiters are interpreted strictly, for example "D-M-Y" the delimiter MUST be "-" in this case. To represent any delimiter use "~".	Script & Assignments
	ToFloat	Converts a string in the specified format to a float number. This is needed, for example, when the money format on imported documents is in a different format to the system.	Script & Assignments

	CreateThereforeObject	This function will create a new Therefore object. As parameter, an array is expected. The first element in the array should always be a string with the object name you want to create. Currently this can be used to create an Index data object. See Creating a New Case . Note: If you use the scripting help dialog, it will insert a predefined CreateThereforeObject call.	Script
Microsoft Word Object	Filename	Gets the file name of the file.	Script & Assignments
	GetControl	Gets the content of an content control. See GetControl for more information.	Script & Assignments
	GetFormField	Gets the content of a form field. Click here for more information.	Script & Assignments
	GetProperty	Gets the content of a document property.	Script & Assignments
	Template	Gets the name of the template used for the document.	Script & Assignments
Microsoft Excel Object	Filename	Gets the file name of the file.	Script & Assignments
	GetCell	Gets the value of the specified cell. See GetCell for more information.	Script & Assignments
	GetCells	Gets the value of each cell in the specified range. Click here for more information.	Script & Assignments
	GetProperty	Gets the content of a document property. See GetProperty for more information.	Script & Assignments
Microsoft Office Object	Filename	Gets the file name of the file.	Script & Assignments
	GetProperty	Gets the content of a document property. See GetProperty for more information.	Script & Assignments

E-mail Profile

Type of Object	Name	Description	Used with
Index Data Object	ExecuteProfile	This function executes a File indexing profile given the indexing profile id and the filepath to the file. Usually used with an e-mail indexing profile, where we can get index data from an attachment too. (e.g PDF attachment).	Script

	GetField	Gets the current index data value of the specified field.	Script
	SetCategory	Gets the category for the current document. You can use an integer (category ID) or a string (category name).	Script
	SetField	Sets the value of the index data field.	Script
	SetCaseNo	Sets the CaseNo for this Index Data object.	Script
	SetCaseDef	Initializes this index data object, with a case definition. DO NOT USE THIS DIRECTLY ON THE PREDEFINED IndexData object (the predefined object is used when you don't write an object name in front of the function call). This should be used in conjunction with CreateThereforeObject. See Creating a New Case.	Script
	CreateCase	Use this after you have initialized the index data object with a case definition and after all mandatory fields are filled in. DO NOT USE THIS DIRECTLY ON THE PREDEFINED IndexData object (the predefined object is used when you don't write an object name in front of the function call). This should be used in conjunction with CreateThereforeObject. See Creating a New Case.	Script
Profile Object	AddNewKeyword	With this function you can add a new keyword to an already existing keyword dictionary.	Script
	FilesToSave	Sets the files that should be saved with this document.	Script
	NotApplicable	Call this function if you want to abort the indexing profile execution.	Script
	ReadPart	Reads a part of the text, specified with a mask. For example to extract 1234 from PO:1234 use: ReadPart (Extract("ExtractedStringName"), "PO*:*"). The "*" represents 'any text' and the "?" represents the section to be extracted.	Script & Assignments
	ScriptError	Stops the execution of the script and throws an error with the specified text.	Script
	ToDate	Converts a string in the specified format to a date.	Script & Assignments

		This is needed, for example, when the date on imported documents is in a different format to the system. Use "D", "M", "Y" for day, month, year respectively. Delimiters are interpreted strictly, for example "D-M-Y" the delimiter MUST be "-" in this case. To represent any delimiter use "~".	
	ToFloat	Converts a string in the specified format to a float number. This is needed, for example, when the money format on imported documents is in a different format to the system.	Script & Assignments
	CreateThereforeObject	This function will create a new Therefore object. As parameter, an array is expected. The first element in the array should always be a string with the object name you want to create. Currently this can be used to create an Index data object. See Creating a New Case. Note: If you use the scripting help dialog, it will insert a predefined CreateThereforeObject call.	Script
Mail Object	AttachmentCount	Gets the number of attachments.	Script & Assignments
	Body	Gets the body of the e-mail.	Script & Assignments
	Categories	In Outlook you can assign categories to an item. The name of the categories assigned to the saved item, can now be retrieved via a script function. Note: this is not supported with the Therefore™ Mail monitoring feature.	Script & Assignments
	CC (Display Name)	Gets the display names of the cc recipients of the e-mail.	Script & Assignments
	CC (SMTP Address)	Gets the smtp addresses of the cc recipients of the e-mail.	Script & Assignments
	From (Display Name)	Gets the display name of sender of the e-mail.	Script & Assignments
	From (SMTP Address)	Gets the smtp address of sender of the e-mail.	Script & Assignments
	GetAttachments	Gets the attachments of the e-mail (full path file names).	Script & Assignments
	Sent	Gets the date the e-mail was sent.	Script & Assignments

	Size	Gets the size of the e-mail.	Script & Assignments
	To (Display Name)	Gets the display names of the recipients of the e-mail.	Script & Assignments
	To (SMTP Address)	Gets the SMTP Addresses of the recipients of the e-mail.	Script & Assignments
	Subject	Gets the Subject of the e-mail.	Script & Assignments

Signed Document Profile

Type of Object	Name	Description	Used with
File Object	File.Created	The creation time of the file.	Script & Assignments
	File.Exists	Returns True if the file exists, otherwise false.	Script & Assignments
	File.Extension	The extension of the file.	Script & Assignments
	File.FullPath	Gets the full path to the processed file.	Script & Assignments
	File.GetPath	Gets the folder name from file ascending. For example, GetPath(0) would return Folder2 for "C:/Folder1/Folder2/File.txt".	Script & Assignments
	File.GetPathFromRoot	Gets the folder name from the root descending. For example, GetPathFromRoot(0) would return Folder1 for "C:/Folder1/Folder2/File.txt".	Script & Assignments
	File.Modified	The last time the file was modified.	Script & Assignments
	File.Name	The full name of the file. To extract part of a file name, e.g. the characters 2 to 6 use Mid(File.Name,2,5).	Script & Assignments
	File.Size	The size of the file in bytes.	Script & Assignments
	File.Title	The title of the file. (e.g: test.docx - > test).	Script & Assignments

Index Data Object	ExecuteProfile	This function executes a File indexing profile given the indexing profile id and the filepath to the file. Usually used with an e-mail indexing profile, where we can get index data from an attachment too. (e.g PDF attachment).	Script
	GetField	Gets the current index data value of the specified field.	Script
	SetCategory	Gets the category for the current document. You can use an integer (category ID) or a string (category name).	Script
	SetField	Sets the value of the index data field.	Script
	SetCaseNo	Sets the CaseNo for this Index Data object.	Script
	SetCaseDef	Initializes this index data object, with a case definition. DO NOT USE THIS DIRECTLY ON THE PREDEFINED IndexData object (the predefined object is used when you don't write an object name in front of the function call). This should be used in conjunction with CreateThereforeObject. See Creating a New Case.	Script
	CreateCase	Use this after you have initialized the index data object with a case definition and after all mandatory fields are filled in. DO NOT USE THIS DIRECTLY ON THE PREDEFINED IndexData object (the predefined object is used when you don't write an object name in front of the function call). This should be used in conjunction with CreateThereforeObject. See Creating a New Case.	Script
Profile Object	AddNewKeyword	With this function you can add a new keyword to an already existing keyword dictionary.	Script
	FilesToSave	Sets the files that should be saved with this document.	Script
	NotApplicable	Call this function if you want to abort the indexing profile execution.	Script

	ReadPart	Reads a part of the text, specified with a mask. For example to extract 1234 from PO:1234 use: ReadPart (Extract("ExtractedStringName"), "PO*:?"). The "*" represents 'any text' and the "?" represents the section to be extracted.	Script & Assignments
	ScriptError	Stops the execution of the script and throws an error with the specified text.	Script
	ToDate	Converts a string in the specified format to a date. This is needed, for example, when the date on imported documents is in a different format to the system. Use "D", "M", "Y" for day, month, year respectively. Delimiters are interpreted strictly, for example "D-M-Y" the delimiter MUST be "-" in this case. To represent any delimiter use "~".	Script & Assignments
	ToFloat	Converts a string in the specified format to a float number. This is needed, for example, when the money format on imported documents is in a different format to the system.	Script & Assignments
	CreateThereforeObject	This function will create a new Therefore object. As parameter, an array is expected. The first element in the array should always be a string with the object name you want to create. Currently this can be used to create an Index data object. See Creating a New Case. Note: If you use the scripting help dialog, it will insert a predefined CreateThereforeObject call.	Script
Data Extractor Object (PDF)	Extract	Gets the value of the defined item name.	Script & Assignments
	ExtractList	Gets the value of the defined item name on all pages of the PDF (only applicable if the item name was defined with Page set to "All pages").	Script & Assignments
	GetFormField	Gets the value of the PDF form field	Script & Assignments

	SetErrorOnMissingTag	If set to True, the extract functions will throw an error if the item name is not found. Otherwise the Extract function will return empty.	Script
PDF Document Properties	GetAuthor	Gets the Author	Script & Assignments
	GetCreationDate	Gets the creation date	Script & Assignments
	GetCreator	Generally gets the application that created the PDF	Script & Assignments
	GetCustomProperty	Gets a custom property from the PDF. e.g. GetCustomProperty("MyProp")	Script & Assignments
	GetKeywords	Gets the keywords associated with the PDF	Script & Assignments
	GetModificationDate	Gets the last modification date	Script & Assignments
	GetProducer	Gets the PDF Producer	Script & Assignments
	GetSubject	Gets the PDF Subject	Script & Assignments
	GetTitle	Gets the PDF Title	Script & Assignments
Data Extractor Object (XML)	Extract	Gets the value of the first tag found. (e.g. Extract("ExampleTag") will extract the value of the first tag with name "ExampleTag").	Script & Assignments
	ExtractAttribute	Gets the value of the specified attribute of the specified tag. (e.g. ExtractAttribute("ExampleTag", "att1") will extract the attribute: att1 from the first tag with name "ExampleTag").	Script & Assignments
	ExtractList	Gets the value of each tag. (e.g. ExtractList("ExampleTag") will extract the values from all tags with name "ExampleTag").	Script & Assignments
	ExtractListAttribute	Gets the value of each attribute in the specified tag. (e.g. ExtractListAttribute("ExampleTag", "att1") will extract the attribute: att1 from all the tags with name "ExampleTag").	Script & Assignments

	ExtractListTable	Similar to ExtractList but ensures that any empty rows in a table are retained. Hence the table tag must be specified. (e.g. ExtractListAttribute("TagName", "TableTag"))	Script & Assignments
	ExtractListXPath	Can be used to execute an XPath query on the XML document.	Script & Assignments
	Count	Gets the count of nodes from the XPath query.	Script & Assignments
	SetErrorOnMissingTag	If set to true, every Extract function will throw an error if the specified tag was not found. Otherwise the Extract function will return "empty". Default is set to false.	Script
Data Extractor Object (Text-Line/Text-Page)	Extract	Gets the value of the defined item name.	Script & Assignments
	ExtractList	Gets the value of the defined item name on all pages of the text file (only applicable if the item name was defined with Page set to "All pages").	Script & Assignments

Advanced Scripts

Find a list of examples of advanced scripts here.

1. Date and Time

1.1 Using ToDate

Overview

We recommend using the ToDate helper function whenever a Date field is assigned. This script should not be used with DateTime fields.

ToDate can be used to be independent of the local computer's date/time settings.

When a date is assigned, the local computer's regional settings are used to configure the date. This can lead to issues while using Therefore™ Online. Use this script to avoid these issues.

Usage Sample

Sample 1:

```
myDate = Helpers.ToDate("2020-01-31", "YYYY-MM-DD")
```

Sample 2:

```
myDate = Helpers.ToDate("31.01.2020", "DD.MM.YYYY")
```

Usually, the first parameter comes from a PDF data extractor:

```
myDate = Helpers.ToDate(Extractor.Extract("..."), "DD/MM/YYYY")
```

1.2 ToDateTime

Use this script to incorporate the language code ID (LCID).

Syntax

```
Helpers.ToDateTime("datetime text", "LCID")
```

Click here for more information to [LCID](#).

Usage sample

The following example converts the date "Mar 19 2021 18:50" to the English - United States date format:

```
Helpers.ToDateTime("Mar 19 2021 18:50", 1033);
```

Output

19.03.2021 18:50

2. Decimals

2.1 ToFloat

Overview

The ToFloat function should be used for decimal numbers.

The format of the function includes "n" and "f".

In this function, "n" stands for the integer portion of the number, and "f" for the decimal places.

- It is important to specify which decimal separator is used (".", ",", "...")
- It is also possible to only allow integer numbers. Leave out "f" to achieve this.

It is recommended to always use ToFloat for decimal numbers.

The standard behavior depends on the Windows regional settings - this might not survive the export to another computer. By always using ToFloat this problem can be avoided.

Usage Sample

```
// using "," as separator  
num = Helpers.ToFloat("499,90", "nn,ff")  
  
// using "." as separator  
num = Helpers.ToFloat("499.90", "nn.ff")  
  
// only allow integer numbers  
num = Helpers.ToFloat("499", "nn")
```

Note:

It does not matter how many n or f are included.

2.2 ToDecimal

Overview

The ToDecimal helper function does exactly the same the [ToFloat](#) function.

In general, we recommend using ToFloat for this task.

When to use ToDecimal:

Use ToDecimal for Decimal fields with a high number of digits. ToFloat won't be able to handle these fields.

For a default Decimal field (15 total digits, 2 decimal digits) ToFloat will always work. Only when working with bigger numbers than the default should ToDecimal be used.

Drawback

Arithmetic operations can not be used with ToDecimal.

E.g.:

```
var a = Helpers.ToFloat("19.99", "nn.ff")
x = a + 10 // works fine
```

```
var b = Helpers.ToDecimal("19.99", "nn.ff")
x = b + 10 // does NOT work -> error
```

The operators +, -, *, / cannot be used with.ToDecimal.
For that purpose, the helper function [Helpers.DecimalAdd](#) was added.

3. Strings

3.1 Regexp

Overview

This regular expression sample script returns the first found match in a string. A similar script without regexp use can be found here: [Get value from string](#).

Usage Sample

```
function GetRegExpMatch(patrn, strng)
{
    var array1 = strng.match(patrn);
    if (array1 === null)
        return undefined;
    else return array1[0];
}
```

How to use this script

Everything between the two "/", will denote the regular expression.

"i" signifies that non case-sensitive search will be executed.

```
myVal = GetRegExpMatch(/\d{4}/i, OCR.Extract("zone1"))
```

3.2 ReadPart

Overview

The ReadPart function can be used to extract a part of a string.

In the "mask" the following special characters are recognized:

- * Matches any character
- ? Start reading characters, until the next character in the mask matches.

Usage Samples

Read everything after: "test:"

```
var result = Profile.ReadPart("test: 12345", "test: ?") // result = "12345"
```

Characters that do not match are ignored.

```
var result = Profile.ReadPart("some characters before - test: 12345",
"test: ?") //result = "12345"
```

However, this causes strange behavior. See this sample:

```
var result = Profile.ReadPart("some characters before t and colon: - test:
12345", "test: ?") // result = "- test: 12345"
```

Why is the result "- test: 12345"?

This is happening because the function matches character by character and ignores characters which are not in the mask.

To solve this issue, we first need to match "test:"

Some characters before t and colon are added: - test: 12345

This did match "test:"- we just ignored the other characters in between. After "test:" we wanted to start reading so we get the rest of the string as a result.

You can see this as a kind of state machine. It always tries to match the next character in the mask.

With ? you "activate" the reading.

You can also "deactivate" the reading again, and activate it later in order to read multiple places.

The following sample is a more advanced example:

```
var result = Profile.ReadPart("some X1234X characters before X5678X and colon",
    "X?XX?X") // result = "12345678"
```

You can read this mask as:

1. Wait for an X
2. Start reading characters
3. Read until another X is found then stop reading.
4. Wait for an X
5. Start reading characters
6. Read until another X is found then stop reading.

Invalid Masks

Any mask containing "**?" or "?*" anywhere is invalid. There will be no error - instead, the full input string is returned.

4. Content Connector and Document Loader

Enter topic text here.

4.1 Setting up a multi-category indexing profile

Overview

This is an indexing profile, which does not have a category set in the Therefore™ Solution Designer. The category is then set via scripting.

Why use such a profile?

It can be used to provide automatic categorization based on (a set of) criteria.

How to setup such a profile?

- Ensure not to set a category in the profile
- Use `IndexData.SetCategory` to set the category via script
- Either use the assignments or the `IndexData.SetField` call to set index data.

Firstly, ensure not to set a category for the profile.

The screenshot shows the 'Indexing Profile' dialog box. The 'Category' field is highlighted with a red rectangle. The dialog contains the following elements:

- Profile name:** multi-category profile
- Filter:** (empty text box)
- Data Extractors:** (empty list box with icons for adding, removing, and sorting)
- Category:** (empty text box, highlighted with a red rectangle)
- Script:** <Multiline script> (with a button to open the script editor)
- Auto-append mode:** Category default (dropdown menu)
- Category Field / Assignment:** (empty table with two columns)
- Buttons:** Test..., ☐ Disable automatic evaluation, OK, Cancel

In the script, assign the correct category based on the relevant criteria:

E.g.:


```
var code = "" // code is coming from Data extractor or file name or...
if (code == "AB")
{
  IndexData.SetCategory("ABCategory")
}
else if (code == "CD")
{
  IndexData.SetCategory("CDCategory")
}
else if (code == "EF")
{
  IndexData.SetCategory("EFCategory")
}
else
{
  No code did match?
```

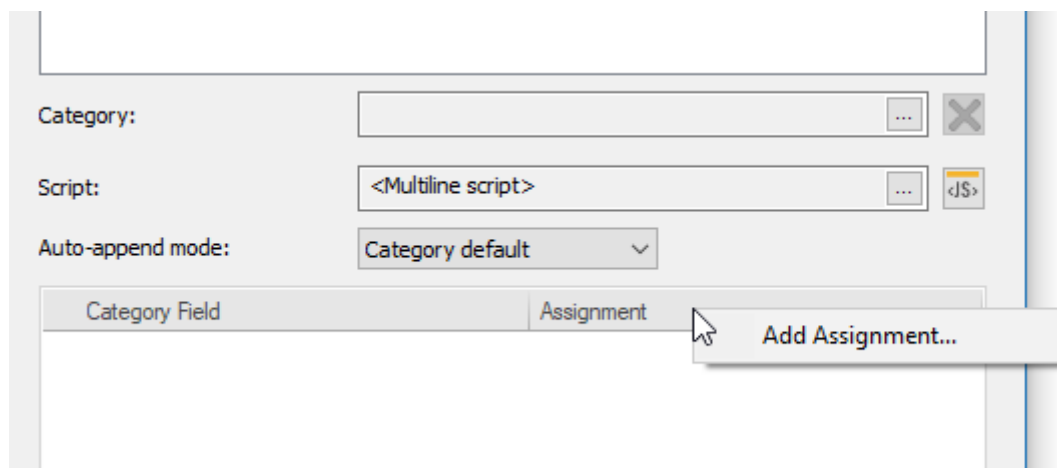
If this is an error for you, you can use:

```
//Profile.ScriptError("No code matched")

// Or maybe send it to a standard category?
//IndexData.SetCategory("Misc");

//Or maybe, in cont conn input folder, try the next profile?
//Profile.NotApplicable()
}
```

Set the index data fields of the selected category. Assignments are not present when a category is chosen. To add assignments, right-click on the assignments header and select "Add assignment..." from the context menu.



Then choose any category or field and the assignment will be added. The executed assignments depend on the chosen category.

E.g.: If `SetCategory("ABCategory")` is set, then only assignments for this category will be executed, all others are ignored.

Script:

Auto-append mode:

Category Field	Assignment
Category: ABCategory	
Text	code ...
Category: CDCategory	
Text	code ...

4.2 FilesToSave

Overview

The FilesToSave property is used to tell what files should be saved for a new document. It can be used in the file indexing profiles for the following Therefore™ applications:

- Content Connector
- Document Loader

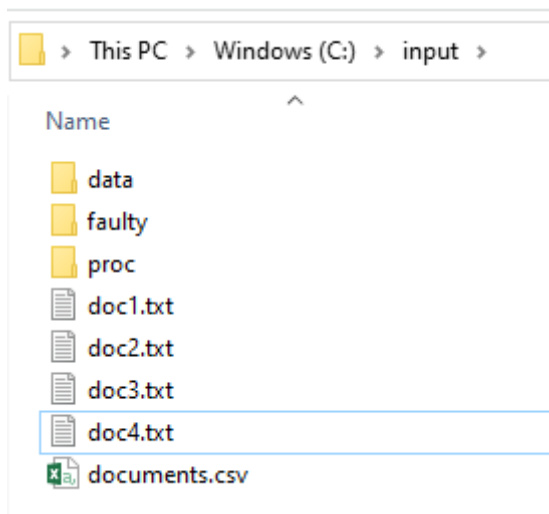
When FilesToSave is not used the processed document is saved by default. For example, if there is a .csv file that generates 10 documents and FilesToSave is not used properly all 10 documents will contain the .csv file itself.

Usage samples

It is possible to specify a relative or an absolute path in the FilesToSave property. Please note that an absolute path cannot be used in Therefore™ Online.

Relative Paths

A relative path is always relative to the input file.



```
Profile.FilesToSave = "doc1.txt"
```

```
// if the files were in the "data" folder it would be this  
Profile.FilesToSave = "data/doc1.txt"
```

```
// often the filename comes from the data extractor - this will save ONE file  
Profile.FilesToSave = Extractor.Extract("filename");
```

It is also possible to save multiple files into one document. In that case, assign an array with all the files the document should contain.

```
// save all 4 documents  
var docs = ["doc1.txt", "doc2.txt", "doc3.txt", "doc4.txt"];  
Profile.FilesToSave = docs;
```

```
// similar when you use a data extractor  
// important is: use the ExtractList function  
Profile.FilesToSave = Extractor.ExtractList("filename")
```

Please note that the FilesToSave property is not cumulative. It will always use what was assigned last.

Absolute Paths

For Therefore™ on-premise, it is also possible to use absolute paths.

```
// backslashes have to be escaped in javascript.  
Profile.FilesToSave = "C:\\input\\data\\doc1.txt"  
  
var docs = ["C:\\input\\data\\doc1.txt", "C:\\input\\data\\doc2.txt"]  
Profile.FilesToSave = docs;
```

Forms

Saving a 'form' - no files, only the index data - can be accomplished as follows:

```
Profile.FilesToSave = undefined;
```

Wildcards

It is also possible to use wildcards to specify a range of documents from a folder. This could be useful if each document that should be saved has a separate folder. Be careful when using a wildcard if documents from the input folder are being saved.

```
Profile.FilesToSave = "data/doc*.txt"
```

4.3 Using the XML Data Extractor

4.3.1 Using XML data extractor

The xml data extractor can be used to read almost any xml file; however several considerations need to be made.

Note:



This guide will focus on the ExtractListXPath function, as this is the most flexible one. This guide also assumes, the normal functions which are proposed in the dropdown menu, do not provide the desired result.

Important considerations before before configuring

- Is 1 xml = 1 document saved in Therefore? Does the xml contain multiple documents which should be saved?
- Is a default namespace used in the xml?
- Are namespaces used in the xml?
- Where are the files that need to be saved?

Configuring the Data extractor - The "Document Tag"

The only thing you need to set is the Document tag. The chosen node depends on this question:

Is 1 xml = 1 document saved in Therefore?

Does the xml contain multiple documents which should be saved?

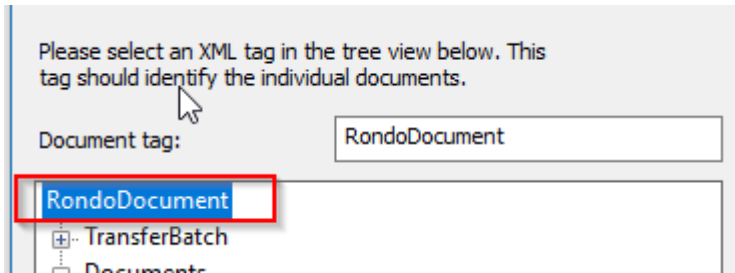
Note:



The Document Tag decides how many documents will be saved.

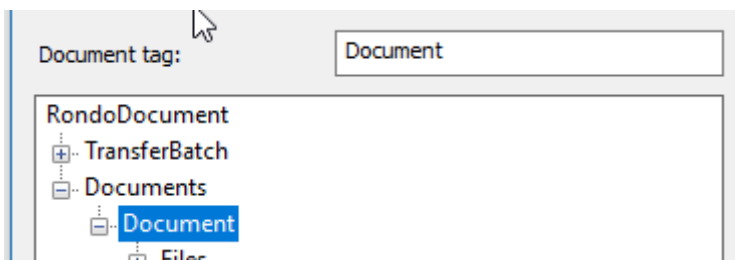
The xml is for one document

Select the first node as the 'Document tag'.



The xml contains multiple documents

Should there be multiple documents saved from the xml file, a tag has to be chosen, which is taken for a document break. The example below contains two documents. For this the "Document" tag can be used.



```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <RondoDocument xsi:noNamespaceSchemaLocation="rondo_document_1_1_0-0_1.xsd" xmlns:xsi="http://www.w3.org
3  <TransferBatch>
10 <Documents>
11   <Document externalId="" status="new" version="">
12     <Files>
15     <Group type="header">
35     <Group type="workflow">
41   </Document>
42   <Document externalId="" status="new" version="">
43     <Files>
46     <Group type="header">
66     <Group type="workflow">
72   </Document>
73 </Documents>
74 </RondoDocument>

```

Note:

This essentially suggests:



- Each <Document> tag found in the xml, will be one Therefore™ document.
- Because the xml contains two <Document> tags, it will be two Therefore documents.
- If you would have chosen the <Group> xml tag, the user would get four Therefore™ documents, because the <Group> tag exists four times in this xml file.

Using the ExtractListXPath function

Assuming that the normal Extract functions, which are available via the dropdown menu do not want or cannot extract the value needed by the user. For more info about ExtractListXPath see also: [XML - ExtractListXPath](#)

The ExtractListXPath function is the most flexible function of all. However, it needs to be distinguished if we only a single document or multiple documents are retained.

The xml is for one document

With only one document, the full XPath expression from the root can be written.

E.g. If the file name is required:

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <RondoDocument xsi:noNamespaceSchemaLocation="rondo_document_1_1_0-0_1.xsd" xmlns:xsi="http://www.w3.org
3  <TransferBatch>
10 <Documents>
11   <Document externalId="" status="new" version="">
12     <Files>
13       <File encoding="ibm850" role="document" type="ascii">V14201904.0042.txt</File>
14     </Files>
15     <Group type="header">

```

This would be:

`Extractor.ExtractListXPath("/RondoDocument/Documents/Document/Files/File")`

Start at the root and go tag by tag:

For more XPath help: `<todo>`

The xml contains multiple documents

Here, the user does not begin at the root. The document tag has been chosen as per the xml data extractor configuration: `<Document>`

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <RondoDocument xsi:noNamespaceSchemaLocation="rondo_document_1_1_0-0_1.xsd" xmlns:xsi="http://www.w3.org
3  <TransferBatch>
10 <Documents>
11   <Document externalId="" status="new" version="">
12     <Files>
13       <File encoding="ibm850" role="document" type="ascii">V14201904.0042.txt</File>
14     </Files>
15     <Group type="header">
35   <Group type="workflow">
41   </Document>
42   <Document externalId="" status="new" version="">
43     <Files>
44       <File encoding="ibm850" role="document" type="ascii">V14201904.0042.txt</File>
45     </Files>
46     <Group type="header">
66   <Group type="workflow">
72   </Document>
73 </Documents>
74 </RondoDocument>

```

To insert the <File> tag, inside the correct <Document>:

Use a dot (".")

The dot tells XPath, to use the current document. The will execute the script twice.

The first time, 1st is the "current" document.

The second time, 2nd is the "current" document.

Because, it begins with the <Document> node, it appear like this:

```
Extractor.ExtractListXPath("./Files/File")
```



Note:

For further XPath help, use: <todo>

Setting FilesToSave

If FilesToSave is not set, the xml itself will be saved.

Usually the xml contains a filename which should be saved instead, that's why Profile.FilesToSave should be set.



Note:

Sample Wiki: <https://support.therefore.net/helpdesk/attachments/77025868690>

4.3.2 ExtractListXPath

The ExtractListXPath function uses XPath, this is a general XML query language which is not specific to Therefore™.

Therefore™ specific function

As an ExtractList function, the function will always return an array.

If only a single value is required, you can simply put [0] or (0) after the function.

```
Extractor.ExtractListXPath("...")[0]
```

XPath - Simple xml tags

When using the XPath, users can go from the xml root, to the value desired.

This is similar to a folder path for files. E.g. in this following xml:

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <ZINVOIC02>
3    <IDOC BEGIN="1">
4      <EDI_DC40 SEGMENT="1">
5        <TABNAM>EDI_DC40</TABNAM>
6        <MANDT>001</MANDT>
7        <DOCNUM>0000000000274024</DOCNUM>
8        <DOCREL>700</DOCREL>
9        <STATUS>30</STATUS>
10       <DIRECT>1</DIRECT>

```

Begin with a forward slash (/) (which indicates the "root"), and continue until arriving at the desired value.

E.g.

`ExtractListXPath("/ZINVOIC02/IDOC/EDI_DC40/DOCNUM")`

This will work for xmls, which contain single documents. For xmls where multiple documents are saved, users should use a dot (.) first to denote the "current" document.

4.3.3 Using automatic categorization with input folders

Overview

When a user intends to use the Therefore™ Content connector to categorize the incoming files based on certain criteria to their respective categories. There are various solutions that can be implemented.

Solution 1

If the files can be easily distinguished by their file name the filter can be used in the input folder.

Below, files starting with "AB" use the "pdf a3 demo invoice" profile, etc.

Input Folder

Folder type: Local Network

Input folder: J:\grohs\Desktop\exporttest\ContentConnectorInputFolder

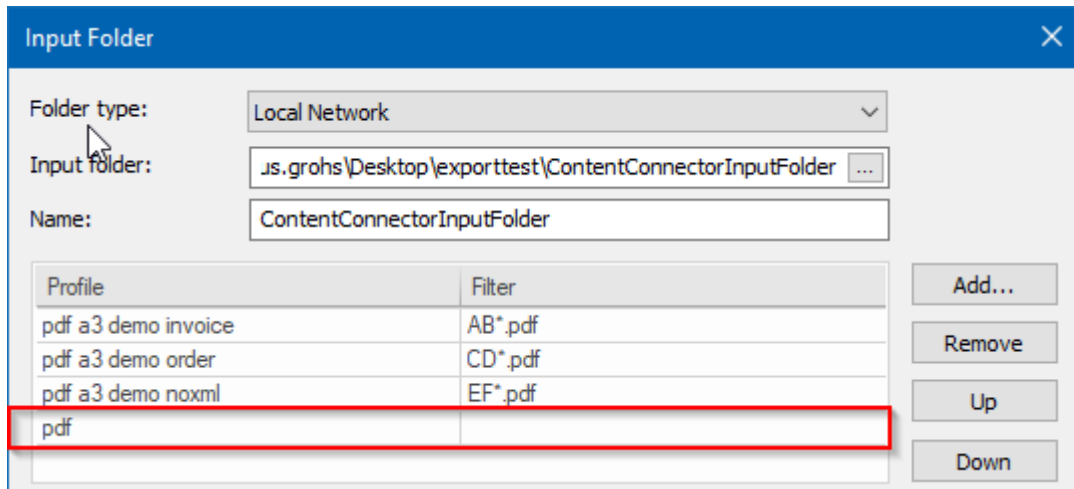
Name: ContentConnectorInputFolder

Profile	Filter
pdf a3 demo invoice	AB*.pdf
pdf a3 demo order	CD*.pdf
pdf a3 demo noxml	EF*.pdf

Add... Remove Up Down

This list is processed from TOP to BOTTOM which can be used to the user's advantage

E.g., when processing files that DO NOT MATCH any of the above the following can be configured:

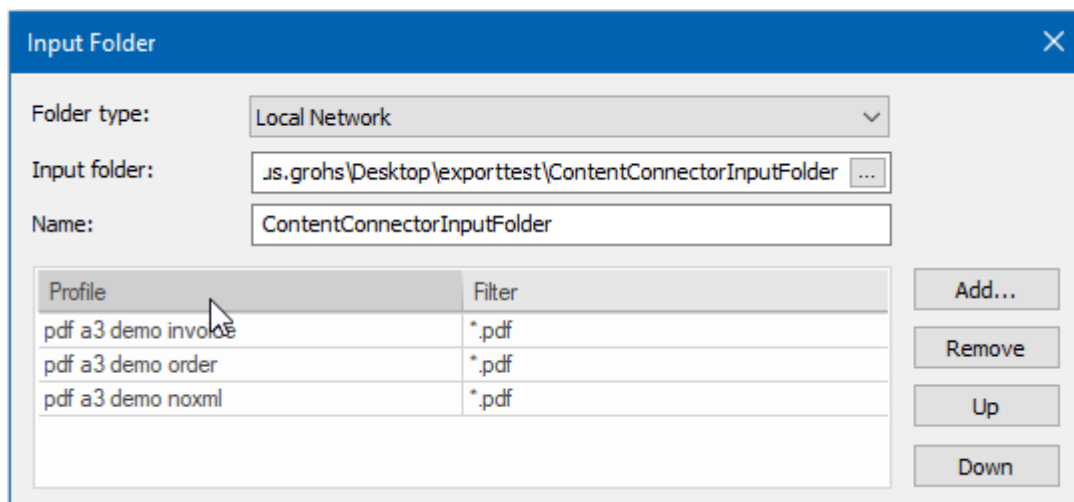


Profile	Filter
pdf a3 demo invoice	AB*.pdf
pdf a3 demo order	CD*.pdf
pdf a3 demo noxml	EF*.pdf
pdf	

Here, the last entry does not have a filter which means it will match any file. This can be used to process files that did not match any of the previous filters.

Solution 2

If Solution 1 with the filter from the input folder is not flexible enough a script can be used to implement a similar task for each indexing profile. The setup will be similar to Solution 1, however, a script can be used to check if a profile is applicable or not.



Profile	Filter
pdf a3 demo invoice	*.pdf
pdf a3 demo order	*.pdf
pdf a3 demo noxml	*.pdf

The filter in the input folder can be left empty. Alternatively, as demonstrated in the screen shot the user can make sure that only PDFs are processed. Using the script, insert a command to check the filter criteria.

```
If (File.Name == "AB.pdf")
{
//we can process the file
}
```

```
else
{
Profile.NotApplicable()
}
```

Note:



- Make sure to call the Profile.NotApplicable() function to exclude non-applicable content. Data extractors can also be used in this setup so that filters can be used more frequently and in a more flexible manner.
- If NotApplicable() is called, the same scenario as the one demonstrated in Solution 1 will occur.

Solution 3

Set up only one indexing profile in the input folder which will enable the selected profile to implement the categorization in script.

Profile	Filter
pdf	

Note:



- A multi-category indexing profile will need to be set up.
- For further information refer to: [How to setup a multi-category indexing profile](#)

Combining Solutions

Solutions can be combined to fit the needs of the user. E.g., Solution 1 can be used for most files. Also, the last profile in the list as seen in Solution 1 could include a profile from Solution 3 which runs everything within the script.

5. Capture client

5.1 Setting up a multi-category indexing profile (2)

What is a multi-category indexing profile?

This is an indexing profile, which does not have a category set in the Therefore™ Solution Designer. The category is then set via scripting.

Why use such a profile?

It can be used to provide automatic categorization based on (a set of) criteria.

How to setup such a profile?

- Ensure not to set a category in the profile
- Use `IndexData.SetCategory` to set the category via script
- Either use the assignments or the `IndexData.SetField` call to set index data.

Firstly, ensure not to set a category for the profile.

The screenshot shows the 'Indexing Profile' dialog box. The 'Category' field is highlighted with a red rectangle. The dialog contains the following elements:

- Profile name:** multi-category profile
- Filter:** (empty text box with a dropdown arrow)
- Data Extractors:** (empty list box with icons for adding, removing, and sorting)
- Category:** (empty text box with a dropdown arrow and a close button, highlighted with a red rectangle)
- Script:** <Multiline script> (with a dropdown arrow and a <VB> button)
- Auto-append mode:** Category default (dropdown menu)
- Category Field / Assignment:** (empty table area)
- Buttons:** Test..., ☐ Disable automatic evaluation, OK, Cancel

In the script, assign the correct category based on the relevant criteria:

E.g.:

```
var code = "" // code is coming from Data extractor or file name or...
if (code == "AB")
{
  IndexData.SetCategory("ABCategory")
}
else if (code == "CD")
{
  IndexData.SetCategory("CDCategory")
}
else if (code == "EF")
{
  IndexData.SetCategory("EFCategory")
}
else
{
  No code did match?
```

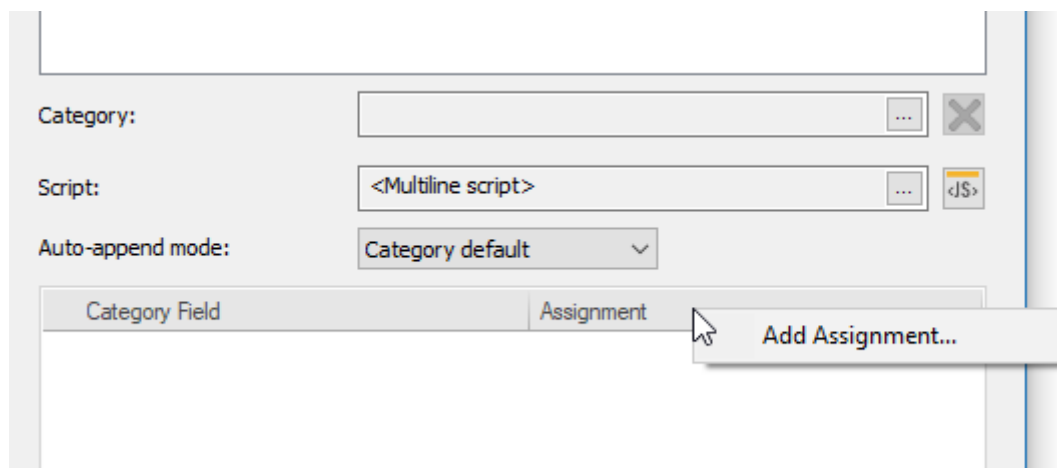
If this is an error for you, you can use:

```
//Profile.ScriptError("No code matched")

// Or maybe send it to a standard category?
//IndexData.SetCategory("Misc");

//Or maybe, in cont conn input folder, try the next profile?
//Profile.NotApplicable()
}
```

Set the index data fields of the selected category. Assignments are not present when a category is chosen. To add assignments, right-click on the assignments header and select "Add assignment..." from the context menu.



Then choose any category or field and the assignment will be added. The executed assignments depend on the chosen category.

E.g.: If `SetCategory("ABCategory")` is set, then only assignments for this category will be executed, all others are ignored.

Script:

Auto-append mode:

Category Field	Assignment	
Category: ABCategory		
Text	code	... v
Category: CDCategory		
Text	code	... v

6. Indexing Profiles

6.1 GetField

Overview

The GetField function will return the index data of an index field. It can be applied in various applications and functions of Therefore™. For example, it can be used in workflow update index data tasks, or in template profiles.

The return value of the GetField function depends on the field type.

Please note that the GetField function only returns index data in the current context. This means it will not load index data from the server.

The GetField is not useful when saving new documents, e.g., in the Therefore™ Content connector, Therefore™ Document Loader, or Microsoft Office add-in. Since saving a new document means there is currently no index data GetField will always return nothing. This is also true even if auto-append would be triggered.

Usage samples

The GetField function has a field identifier as its only parameter. The field identifier should uniquely identify an index field.

Usually, this is simply the FieldId of the field. This is the recommended usage.

It is also possible to use the field GUID as the identifier.

Please note that the object name may be different depending on the application you use are using the GetField function in.

For example, the object name could be:

IndexData.GetField

SourceIndexData.GetField

All following samples use IndexData as the object name.

GetField using FieldId

The 'Invoices' field list on the left shows fields: Invoice No, Invoice Date, Invoice Amount (net), Invoice Amount (incl. VAT), Customer Name, and Customer Address. The 'Field Properties' dialog box on the right shows the 'Field' tab with the following details:

- Field type: Text
- Caption: Invoice No
- Field ID: Invoice_No (highlighted in red)
- Length: 50
- Database index: None
- Default:
- ☒ Mandatory
- ☒ Visible
- ☐ Case sensitive
- ☐ Create label field
- ☐ Drop-down box
- ☐ Drop down during typing
- ☐ Sort descending

Information box at the bottom of the dialog:

This field can be referenced using field number '5355'.
Unique ID: FE0F9485-A866-4C8C-A711-D7E96FEF1700
The column in the index data table is 'Invoice_No'.

```
var data = IndexData.GetField("Invoice_No")
```

GetField using Field GUID

The 'Invoices' field list on the left is the same as in the previous screenshot. The 'Field Properties' dialog box on the right shows the same details, but the 'Unique ID' is highlighted in red in the information box at the bottom:

This field can be referenced using field number '5355'.
Unique ID: FE0F9485-A866-4C8C-A711-D7E96FEF1700 (highlighted in red)
The column in the index data table is 'Invoice_No'.

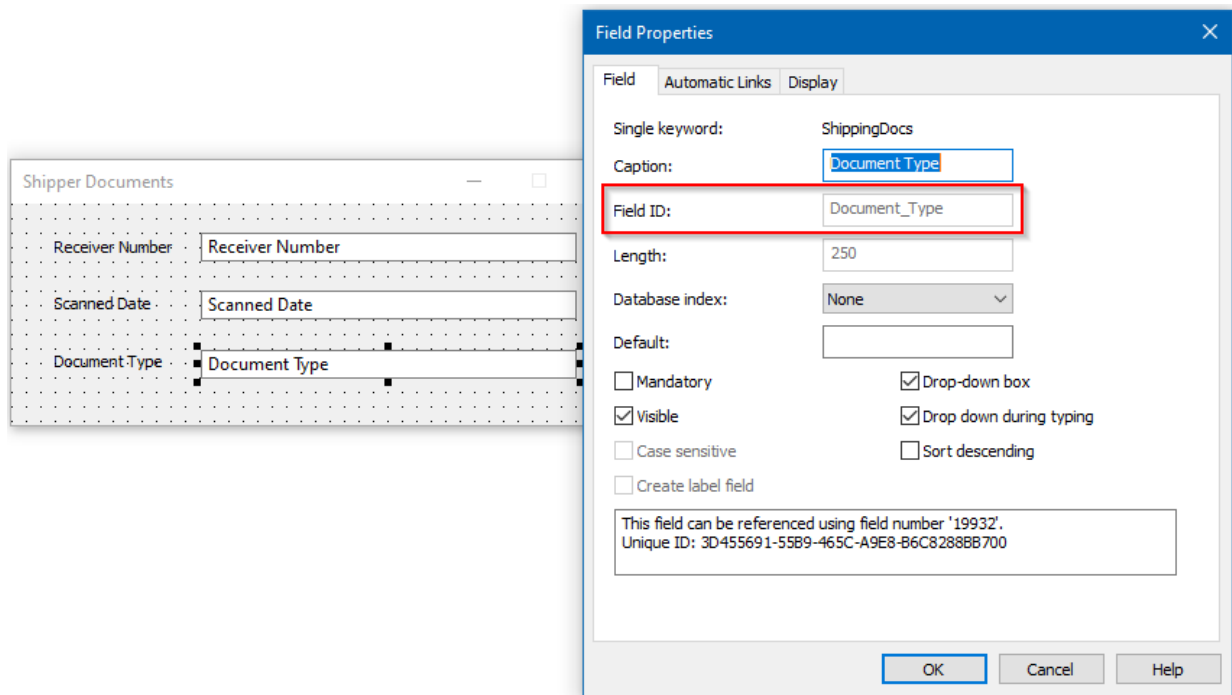
```
var data = IndexData.GetField("FE0F9485-A866-4C8C-A711-D7E96FEF1700")
```


Single Keyword Fields

Single keyword fields are a special case since they technically consist of two fields:

- An ID field containing the keyword ID, and
- a text field containing the keyword text.

To get the keyword ID use the Field ID as shown in the below screenshot using the Therefore™ Solution Designer.



```
var id = IndexData.GetField("Document_Type")
```

To get the text of the single keyword please use the Field ID and append "_Text":

```
var text = IndexData.GetField("Document_Type_Text")
```

Return Values based on Field Type

The return value of GetField depends on the index data field type. The JavaScript value undefined is returned for index data fields in which the value is currently not set.

A string is returned for the following field types:

- Text field
- Formatted counter field
- Single keyword text field
- Primary/Dependent fields based on the database type.

A number is returned for the following field types:

- Number field
- Numeric counter field
- Decimal field

- Checkbox/Logical field
- Single keyword ID field

Let's differentiate what number means. In this context, number refers to the JavaScript number type. JavaScript uses the same type for integer and floating-point types.

For Number fields, Numeric counter fields, and Checkbox/Logical fields the output will always be an integer. For Checkbox/Logical fields, the output can be one of the following three values:

- 0 - unchecked
- 1 - checked
- 2 - undetermined

Decimal fields can be a number or they can be a special Decimal type. This depends on the configuration of the field.

Field Properties

Field Automatic Links Display Calculation

Field type: Decimal

Caption: Invoice Amount (net)

Field ID: Invoice_Amount_net

Total digits: 15

Decimal digits: 2

Database index: None

Default:

☐ Mandatory ☐ Drop-down box

☒ Visible ☐ Drop down during typing

☐ Case sensitive ☐ Sort descending

☐ Create label field

This field can be referenced using field number '5357'.
Unique ID: 8CA26F63-BAAF-4387-941D-1A4537624990
The column in the index data table is 'Invoice_Amount_net'.

OK Cancel Help

By default, a decimal field is configured as seen in the screenshot above. Such fields are returned as number.

If the "Total digits" are increased to over 15 the Decimal field can hold longer numbers, and a special Decimal type will be returned. This is necessary because the JavaScript number type is no longer capable of holding the bigger Decimal type.

This also means that when a special Decimal type is returned you can no longer do standard arithmetic operations such as plus/minus/multiply/divide. Instead, use our Helper methods for Decimal such as 'Helpers.DecimalAdd', etc.

A [JavaScript date object](#) is returned for the following field types:

- Date field
- Date/Time field

For date fields the time should be ignored. Use the 'UTC' functions on the date object to get the date as seen in the following examples:

- getUTCMonth()
- getUTCFullYear()
- getUTCDate()

For date/time fields, when using the "local time" getters such as getHours() they are in local time based on the computer that executes the script!

An array of strings is returned for a multiple keyword field. The array contains each selected keyword name.

Note:



If an array is assigned to a normal field only the FIRST value will be taken. An example could be a template profile. If an assignment such as GetField("myMultiKeywordField") is used only the first value will be used. If all values should be taken, please create a comma separated list manually as shown in the example below.

```
// create a comma separated list of the keyword names  
var list = GetField("myMultiKeywordField").join(",")
```

For tables, an array of the specific field type is returned.
For example:

- For text columns, an array of strings is returned.
- For number columns, an array of numbers is returned,
- and so on.

An empty value for a row will be returned as undefined. If the whole table is empty then an array with no entries will be returned (array.length == 0).

6.2 SetField

Overview

The SetField function can be used to set a field using a script. Functionally, it works the same way as using an assignment in the indexing profile. SetField is also used in the Create case script because there is a second separate IndexData object and there are no assignments for it in the UI.

General usage

SetField has two parameters and does not return anything.

- The first parameter is the field identifier.

The field identifier should uniquely identify an index field. Usually, this is simply the FieldId of the field. It is also possible to use the field GUID as the identifier.

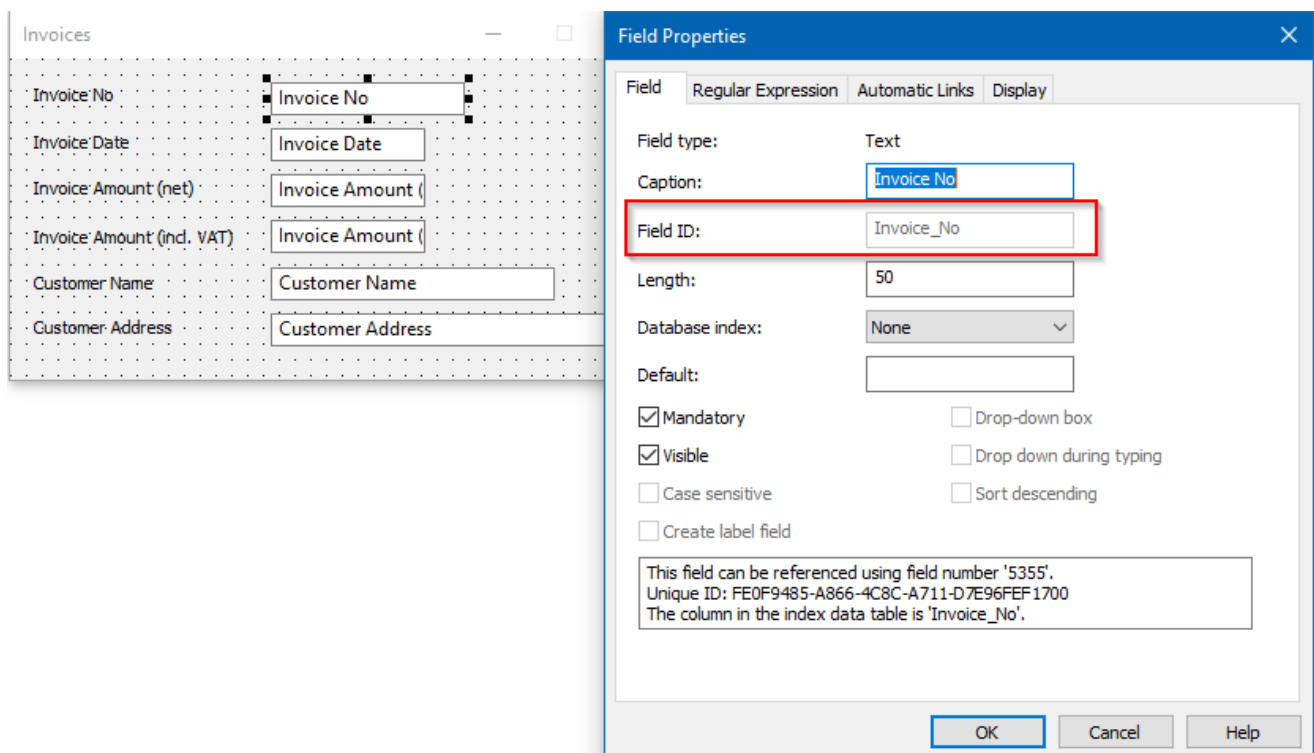
- The second parameter is the new value for the index data field.
Allowed values for this parameter depend on the index data field type.

Please note that the object name may be different depending on where it is being used.
For example, it could be:

- IndexData.SetField
- IxDataCaseHeader.SetField

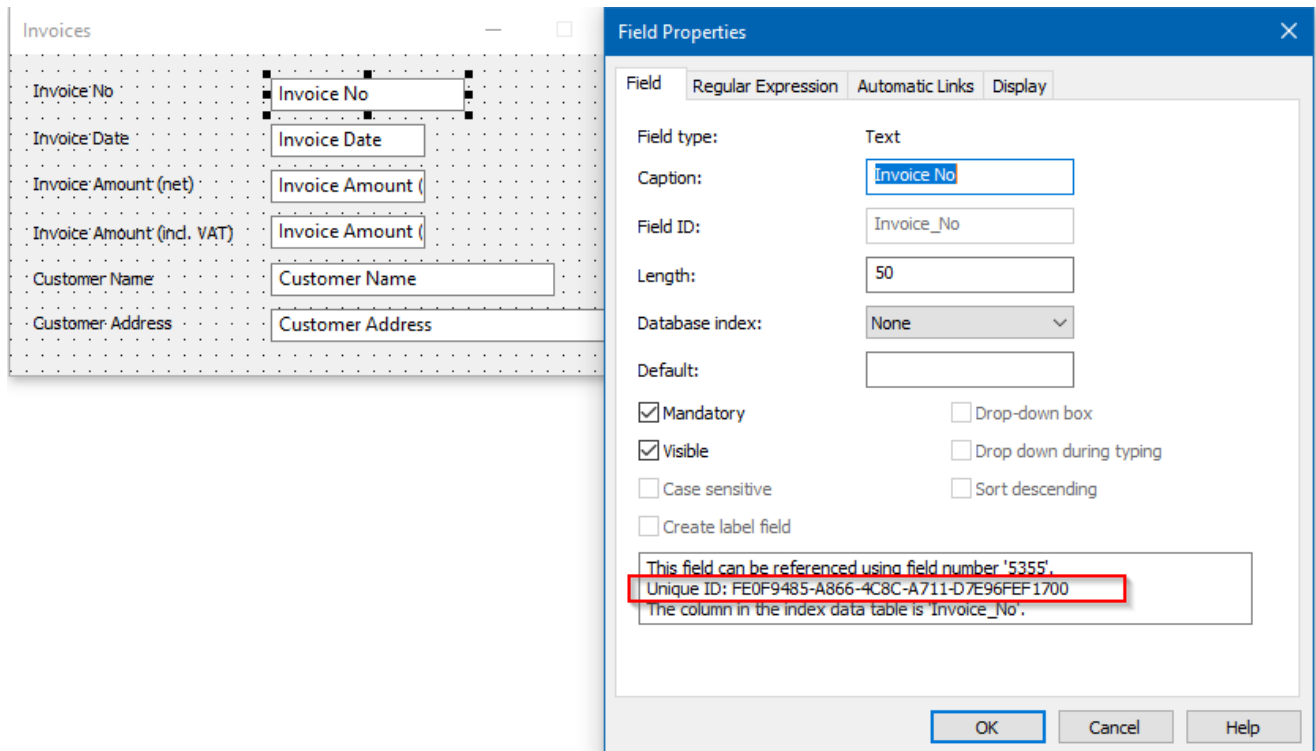
All of the following samples use IndexData as the object name.

SetField using Field ID



```
IndexData.SetField("Invoice_No", "INV12345");
```

SetField using Field GUID



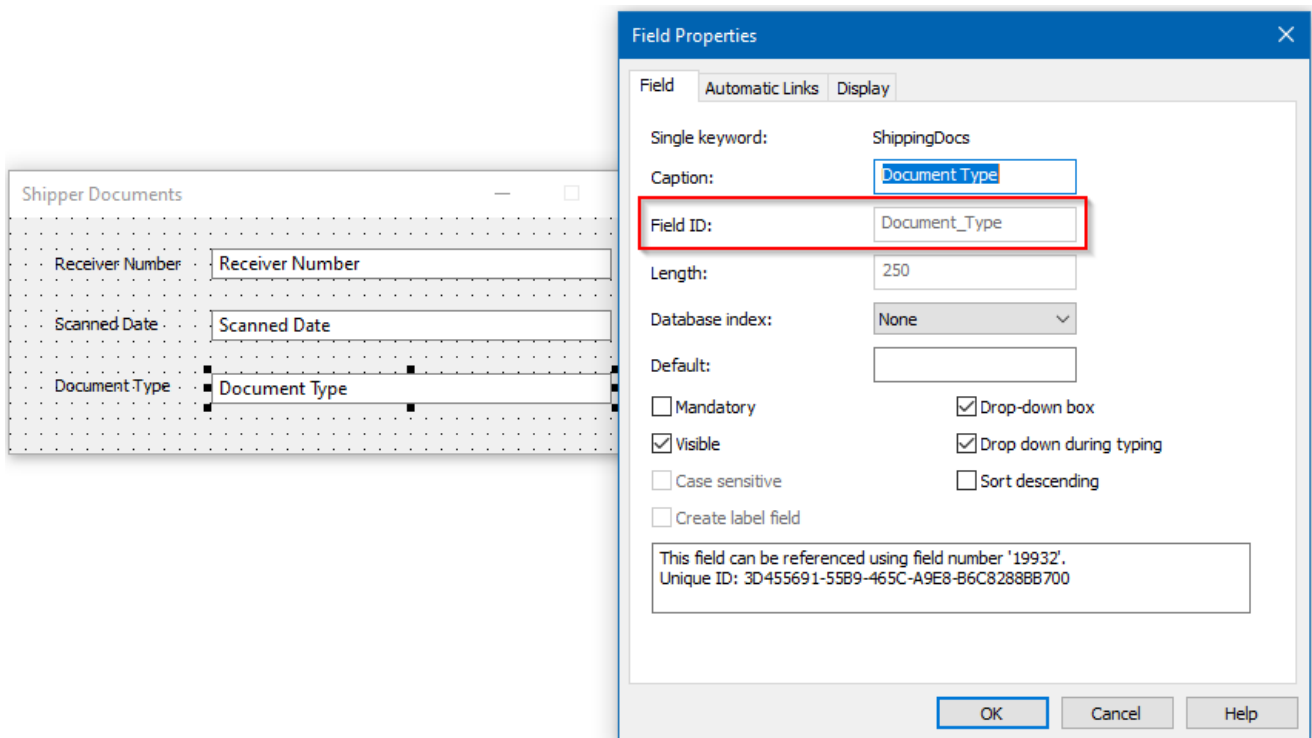
```
IndexData.SetField("FE0F9485-A866-4C8C-A711-D7E96FEF1700", "INV12345")
```

Single Keyword Fields

Single keyword fields are a special case because technically they consist of two fields:

- An ID field containing the keyword ID and
- a text field containing the keyword name.

To set the keyword ID use the Field ID as shown in the below example using the Therefore™ Solution Designer.



```
IndexData.SetField("Document_Type", 1);
```

To set the name of the single keyword use the Field ID and append "_Text".

```
IndexData.SetField("Document_Type_Text", "Invoice");
```

It is recommended to only set one of the two fields. Set either the keyword ID or the keyword name but not both.

Allowed Values based on Field Type

Text Field

A string should be passed.

```
IndexData.SetField("Test", "my new field value");
```

Number Field

A number should be passed.

```
IndexData.SetField("Test", 42);
```

Decimal Field

A number or the special Decimal type should be passed. These values can only be obtained through using [GetField](#) or [ToDecimal](#).

```
// for decimal numbers always use "." as comma separator.
```

```
IndexData.SetField("Test", 42.42);
```

```
IndexData.SetField("Test", Helpers.ToFloat("42,42", "nn,ff"));
```

```
IndexData.SetField("Test", Helpers.ToDecimal("42,42", "nn,ff"));
```

```
// both achieve the same thing but in general using ToFloat is recommended.
```

It is recommended to always use the [ToFloat](#)/ToDecimal helper function when converting a string to a number.

Checkbox/Logical Field

A number or true/false should be passed.
Numbers have the following meanings:

- 0 - unchecked
- 1 - checked
- 2 - undetermined

Any other number is interpreted as 1 - checked.

```
IndexData.SetField("Test", 0); // unchecked
IndexData.SetField("Test", false); // unchecked

IndexData.SetField("Test", 1); // checked
IndexData.SetField("Test", true); // checked
IndexData.SetField("Test", 42); // checked

IndexData.SetField("Test", 2); //undetermined

// the following is not really correct but works too
IndexData.SetField("Test", "false"); // unchecked
IndexData.SetField("Test", "true"); // checked
// also case-insensitive
IndexData.SetField("Test", "TrUe"); // checked
```

Date Fields

A JavaScript date object should be passed. Usually, a date will be available in string format. In that case, use the [ToDate](#) helper function.

```
IndexData.SetField("Test", Helpers.ToDate("31.01.2020", "DD.MM.YYYY"));

// for "today" use
IndexData.SetField("Test", Helpers.CurrentDate());
```

DateTime Fields

A JavaScript date object or a string containing an ISO 8601 date/time value should be passed. Usually, the date/time will be available in string format. In case of an ISO 8601 date/time value it can be used directly as string. Otherwise, use the [ToDateTime](#) helper function.

```
//ISO date/time values
IndexData.SetField("Test", "2022-03-29T10:05:45Z") // UTC time

IndexData.SetField("Test", "2022-03-29T10:05:45-06:00") // Timezone -06 hours

IndexData.SetField("Test", Helpers.ToDateTime("Mar 19 2021 18:50", 1033))
```

Numeric Counter Fields

A numeric counter field follows the same rules as a Number Field. In general, it is recommended not to set it since it is being set automatically.

Formatted Counter Fields

A formatted counter field follows the same rules as a Text Field. In general, it is recommended not to set it since it is being set automatically.

Single Keyword Fields

- The single keyword ID field follows the same rules as a Number Field.
 - The single keyword Name field follows the same rules as a Text Field.
- Additionally, it is also checked if the keyword exists which means it is not possible to set an invalid ID or an invalid keyword name.

```
// set keyword ID
IndexData.SetField("Document_Type", 1);

// set keyword Name
IndexData.SetField("Document_Type_Text", "Invoice");
```

It is recommended to only set one of the two fields. Either set the keyword ID or the keyword name but not both.

Multiple Keyword Fields

These contain either a comma separated list of entries or an array of entries. The keywords can only be set by name. It is not possible to set the keywords by ID. For each keyword, it is checked if it exists in the keyword dictionary.

```
IndexData.SetField("Departments_MKW", "IT, Legal")

var list = ["IT", "Legal"]
IndexData.SetField("Departments_MKW", list)
```

Primary/Dependent Fields

The allowed values depend on the column type in the database for these fields. In general, there are the same types as for normal fields, e.g., Text Field, Number field, etc. Only the primary field must be set up as the primary field should uniquely identify the row in the database.

Fields in a Table

A table allows the same values as a normal field. The only difference is that the values should be in an array. Each array entry is the value of the column in one row.

	Product (Text)	Quantity (Number)	Total (Number)
Row 1			
Row 2			
Row 3			

```
var products = ["Coffee", "Snacks", "Beer"];
```



```
IndexData.SetField("Product", products);
```

The result will be:

	Product (Text)	Quantity (Number)	Total (Number)
Row 1	Coffee		
Row 2	Snacks		
Row 3	Beer		

Use undefined if you want to leave a row empty.

```
var products = ["Coffee", "Snacks", "Beer"];
IndexData.SetField("Product", products);

var quantity = [42, undefined, 3];
IndexData.SetField("Quantity", quantity);
```

The result will be:

	Product (Text)	Quantity (Number)	Total (Number)
Row 1	Coffee	42	
Row 2	Snacks		
Row 3	Beer	3	

If more rows are added for one column the other columns will be automatically empty.

```
var products = ["Coffee", "Snacks", "Beer"];
IndexData.SetField("Product", products);

var quantity = [42, undefined, 3];
IndexData.SetField("Quantity", quantity);

// we add 4 items here!
var total = [ 42, 0, 3, 50];
IndexData.SetField("Total", total);
```

	Product (Text)	Quantity (Number)	Total (Number)
Row 1	Coffee	42	42
Row 2	Snacks		0
Row 3	Beer	3	3
Row 4			50

It is important to note that it is not possible to remove rows from a table when the index data is updated and there already is a filled table for example in the [Update Index](#) workflow task. Basically, with SetField alone one can only grow a table by adding rows.

Adapt the following script to remove rows from a table. In this example, 2 rows will be removed from a 4 row table.

```
// get the value of all columns of the table
var products = IndexData.GetField("Product");
var quantity = IndexData.GetField("Quantity");
var total = IndexData.GetField("Total");

// delete the whole table
IndexData.SetField("Table1", undefined);

// remove every entry except the first two.
while(products.length > 2)
{
    products.pop()
}
while(quantity.length > 2)
{
    quantity.pop()
}
while(total.length > 2)
{
    total.pop()
}

// set columns to table
IndexData.SetField("Product", products);
IndexData.SetField("Quantity", quantity);
IndexData.SetField("Total", total);
```

	Product (Text)	Quantity (Number)	Total (Number)
Row 1	Coffee	42	42
Row 2	Snacks		0

The important part is deleting the table before setting it again. See also [Delete All Entries](#).

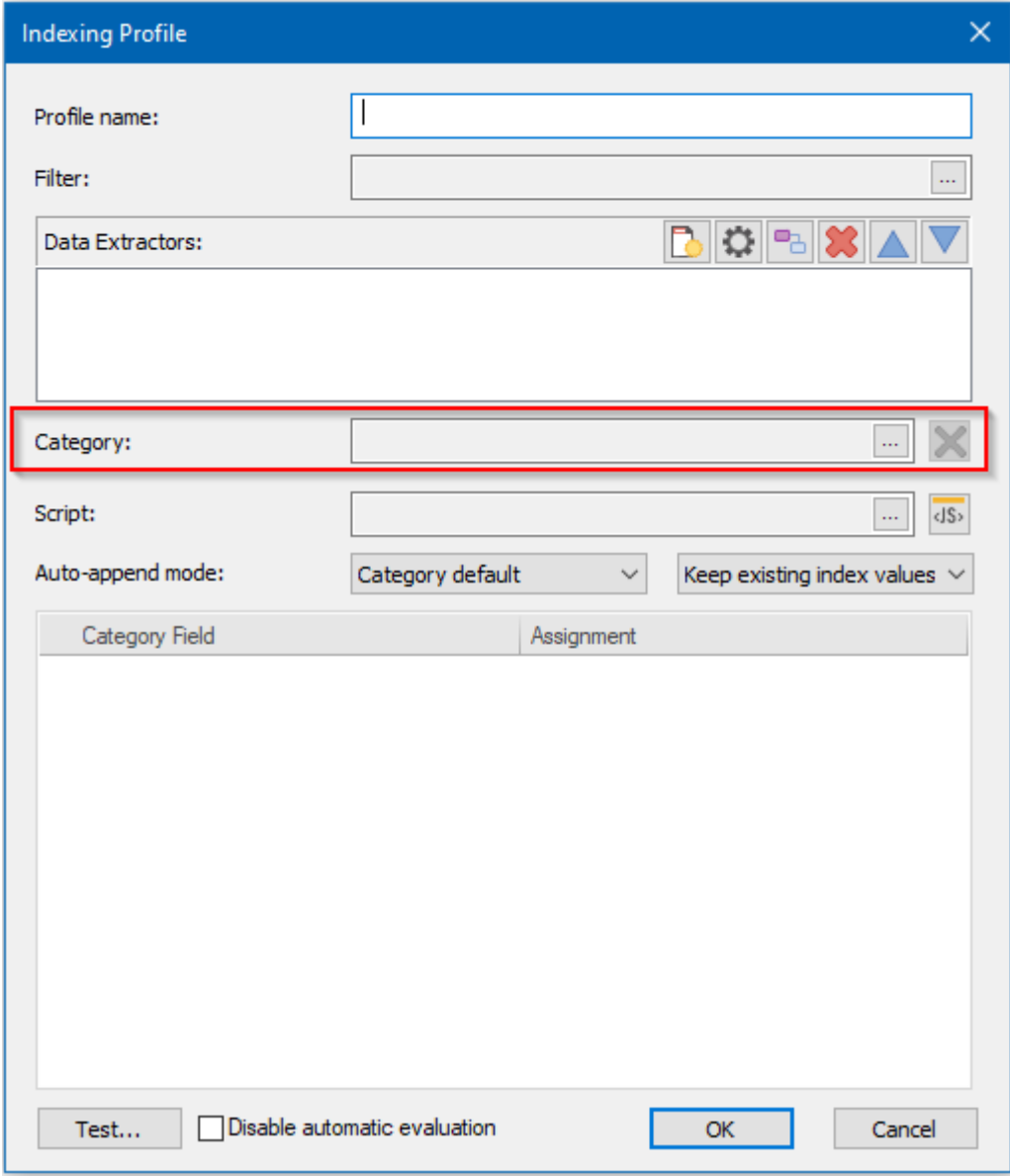
6.3 SetCategory

Overview

The SetCategory function can be used to dynamically set the category of an indexing profile at runtime. The general use case is to use it if the profile itself decides in which category a document should be saved to. This could for example be a barcode - based on the barcode value a category is chosen.

General setup

Leave the Category selection in the indexing profile empty if the use of SetCategory is intended. This also applies while using SetCategory in the Therefore™ Capture client.

The image shows a software dialog box titled "Indexing Profile". It contains several input fields and controls. At the top, there is a "Profile name:" text box. Below it is a "Filter:" text box with a browse button (...). The "Data Extractors:" section has a list box and a toolbar with icons for adding, configuring, deleting, and sorting. The "Category:" section is highlighted with a red rectangle; it contains a text box, a browse button (...), and a delete button (X). Below this is a "Script:" text box with a browse button (...) and a code icon (<JS>). The "Auto-append mode:" section has two dropdown menus: "Category default" and "Keep existing index values". At the bottom, there is a table with two columns: "Category Field" and "Assignment". The table is currently empty. At the very bottom of the dialog are buttons for "Test...", a checkbox for "Disable automatic evaluation", "OK", and "Cancel".

The next step is to develop the script which is calling the SetCategory function.

The SetCategory function can be used in 3 ways.

- By using the name of the category (recommended)
- By using the Guid of the category
- By using the ID of the category (not recommended)

There is currently no easy way to show the Guid of a category except for looking into the database.

```
var value = "AB"; //TODO: get value from somewhere, for example from a barcode.  
if (value == "AB")  
{
```

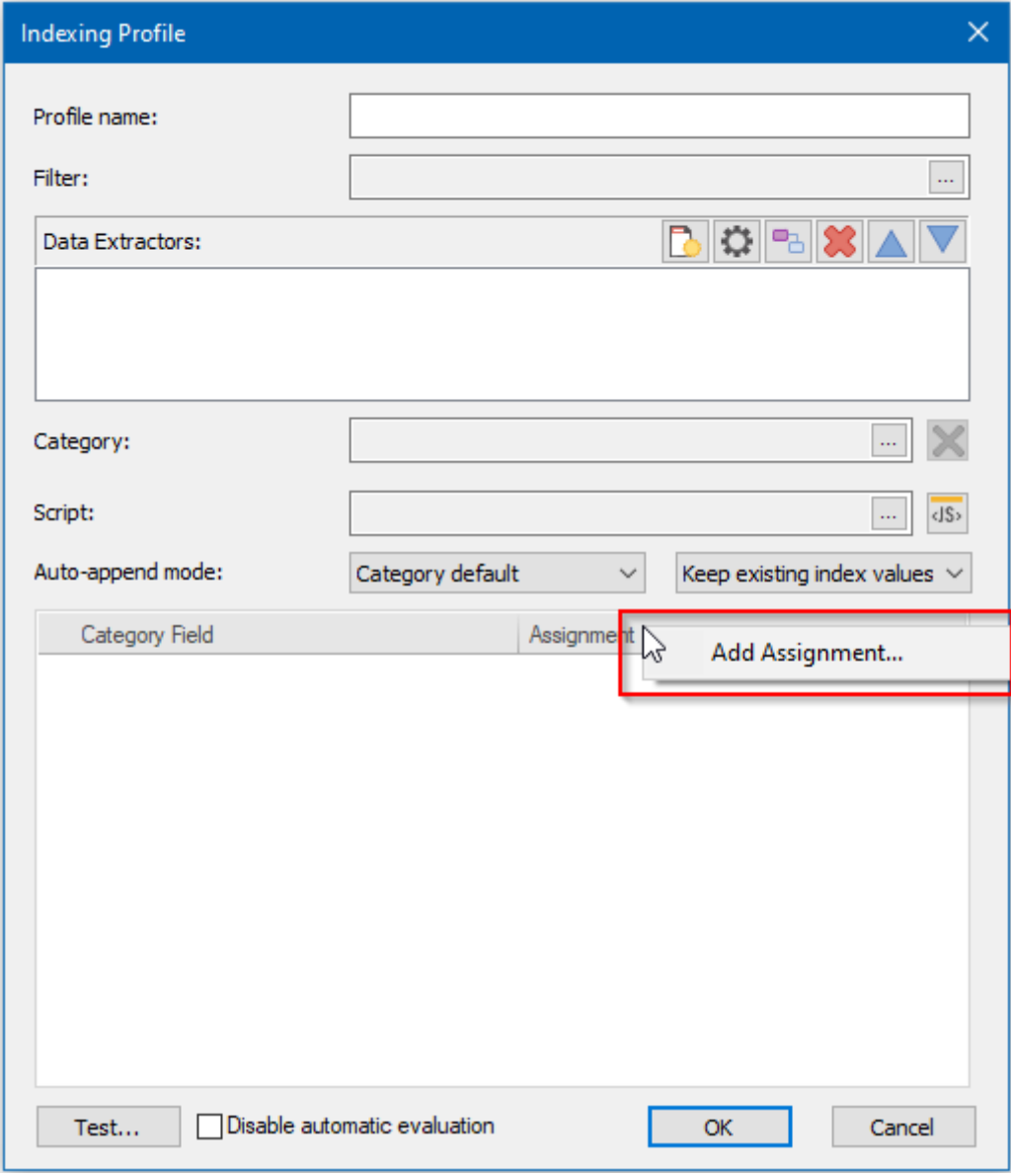
```

    IndexData.SetCategory("Invoices");
}
else if (value == "CD")
{
    IndexData.SetCategory("Contracts")
}
else if (value == "EF")
{
    IndexData.SetCategory(1); // IDs are not recommended as they change during
import/export.
}
else
{
    IndexData.SetCategory("18AD2A06-2A3F-4644-92BA-C8FF36414D76");
}

```

The final step is setting the index data fields. This can be accomplished using the [SetField](#) function.

Alternatively, right-click on the Assignment header to choose multiple categories for the assignments. Only the assignments for the set category will be executed. All other assignments are ignored.



The image shows a software dialog box titled "Indexing Profile". It contains several input fields and controls:

- Profile name:** A text input field.
- Filter:** A text input field with a dropdown arrow.
- Data Extractors:** A list box with icons for adding, configuring, deleting, and sorting.
- Category:** A text input field with a dropdown arrow and a close button.
- Script:** A text input field with a dropdown arrow and a "JS" icon.
- Auto-append mode:** Two dropdown menus, one set to "Category default" and the other to "Keep existing index values".
- Assignment Table:** A table with two columns: "Category Field" and "Assignment". A red box highlights the "Add Assignment..." button next to the "Assignment" column header.
- Buttons:** "Test...", "Disable automatic evaluation" (checkbox), "OK", and "Cancel".

For a related topic, see also [How to setup a multi-category indexing profile](#).

7. Workflow

7.1 Adding Rows

Overview

This script can be used in the Update Index Data Workflow Task.

This script is useful for you if the following conditions apply:

- You have a table that already contains some rows (or is empty)
- You do not want to change any existing rows
- You want to add a new row at the end of the table

An example would be:

	First Column	Second Column	Third Column
Row 1	1	4	7
Row 2	2	5	8
Row 3	3	6	9

If you want to add a new row, Row 4, you have to:

- Get all existing columns of the table where a value should be set.
- Increase the existing array by 1 (or by the number of rows you want to add)
- Set the new value in the array.
- Save the array back into the index data.

Usage Samples

Get the existing index data of the table columns.

```
first = SourceIndexData.GetField("First Column")
second = SourceIndexData.GetField("Second Column")
third = SourceIndexData.GetField("Third Column")
```

Set the new values for the new row.

```
first[3] = "X"
second[3] = "Y"
third[3] = "Z"
```

Save back all changed arrays either by using the SetField call, or by using the variables (first, second, third) in the assignments.

If you want to leave a column unset (blank), then assign undefined.

The resulting table would look like this:

	First Column	Second Column	Third Column
Row 1	1	4	7
Row 2	2	5	8

Row 3	3	6	9
Row 4	X	Y	Z

7.2 Updating Rows

Overview

This script can be used in workflow tasks, for example in the Update Index Data Workflow Task. This script can not be used in the content connector, document loader, or e-forms.

Usage Samples

Generally you can access each column of the table separately, and get an array back for all rows.

When to use this script:

- The table already has table rows
- You do not want to delete table rows
- You do not want to add table rows
- You want to change an already existing value.

Existing index data is needed for this script to work.

In this example, we want to update Row 1 of the first column from 1 to 42.

	First Column	Second Column	Third Column
Row 1	1	4	7
Row 2	2	5	8
Row 3	3	6	9

First, we need to get an array of the first column:

```
first = SourceIndexData.GetField("First Column")
```

This will get you an array with the following content.

Note that Arrays always start at position 0, not at position 1.

First	Array Content
Array Position 0	1
Array Position 1	2
Array Position 2	3

Because we want to change Array position 1, which has the content "2", to "42", we will have to do it like this.

```
first(1) = 42
```

After changing, it is of course important to save the changes back. This means either use the [SetField](#) function or use first in the indexing profile assignments for this table column.

```
first = SourceIndexData.GetField("First Column")
first[1] = 42
IndexData.SetField("First Column", first)
```

Here, you can also use first in the assignments.

7.3 Delete all Entries

Usage Sample

Use the following line of code to delete all entries in a table.

```
IndexData.SetField("Table field Id", undefined)
```

7.4 Delete one Row

Overview

When deleting a single row (or more) from a table field, the following should be implemented:

1. Collect ALL columns from this table and save it to variables.
2. Delete the whole table content.
3. Save back only the entries that are intended to be kept.

Applicability

This only makes sense where an existing document exists. As such, it does not apply to Content Connector/Document Loader or anywhere where the creation of new document occurs (e.g. eForms).

In other words, this means it is most useful for the workflow "Update Index" task.

Usage Sample

Lets assume we have a category with a table field.
The table field has 3 columns named:

- Text1
- Text2
- Text3

1. Beginning with (1), Collect ALL columns from this table and save it to variables.


```
var text1 = SourceIndexData.GetField("Text1")
var text2 = SourceIndexData.GetField("Text2")
var text3 = SourceIndexData.GetField("Text3")
```

2. Then delete the existing content of the table. The values that are no longer wanted then have to be deleted from each array.

```
IndexData.SetField("Table field Id", undefined)
```



Delete the table on "IndexData" and NOT on "SourceIndexData". This is because only the "IndexData" object is being saved and should contain the new values.

E.g.:

Text1	Text2	Text3
a	b	c
delete	e	f
g	h	i

Delete the row where 'Text1' is "delete". Hence, loop over the text1 column and check if it is "delete", and if so, continue with the process. If it is not "delete" then copy over the full row content for every column!

```
var text1New = []
var text2New = []
var text3New = []
for (var i = 0; i < text1.length; i++)
{
  if (text1[i] == "delete")
  {
    // we found the row we want to delete.
    // do NOT copy the row over, just continue
  }
  else
  {
    // not the row we want to delete - copy over
    // we have to do this FOR EVERY COLUMN.
    text1New.push(text1[i])
    text2New.push(text2[i])
    text3New.push(text3[i])
  }
}
IndexData.SetField("Text1", text1New);
IndexData.SetField("Text2", text2New);
IndexData.SetField("Text3", text3New);
And finally we save back the new array with the SetField function.
```

Full Sample

```
var text1 = SourceIndexData.GetField("Text1")
var text2 = SourceIndexData.GetField("Text2")
var text3 = SourceIndexData.GetField("Text3")
IndexData.SetField("Table1", undefined)
var text1New = []
var text2New = []
var text3New = []
for (var i = 0; i < text1.length; i++)
{
  if (text1[i] == "delete")
  {
    // we found the row we want to delete.
    // do NOT copy the row over, just continue
  }
  else
  {
    // not the row we want to delete - copy over
    // we have to do this FOR EVERY COLUMN.
    text1New.push(text1[i])
    text2New.push(text2[i])
    text3New.push(text3[i])
  }
}
IndexData.SetField("Text1", text1New);
IndexData.SetField("Text2", text2New);
IndexData.SetField("Text3", text3New);
```

Result

Text1	Text2	Text3
a	b	c
g	h	i

7.5 Document Creation Workflow Tasks

Overview

The [Add File to Document](#) and [Create New Document](#) tasks in the Therefore™ Workflow can create new files based on an existing PDF, Microsoft Word, or Microsoft Excel Template and also fill them with index data from the source category.

The 'Add File to Document' task adds such a file to an existing Therefore™ document. The file is always added to the current Therefore™ Workflow document.

The 'Create New Document' task creates a new Therefore™ document, and adds the generated file. For this task, file generation is optional; it can also create an empty Therefore™ document. It is also not limited to the source document or Therefore™ Workflow: new documents can be generated in any category.

Document Creation

Document creation is similar for both tasks, and only works with PDF, Microsoft Word, or Microsoft Excel-Templates. Depending on the chosen template type different field-types are available to be filled with index data.

Field Types in Word

- Content Controls
 - o Rich Text Content Control
 - o Plain Text Content Control
 - o Checkbox Content Control
 - o Combo Box Content Control
 - o Drop-Down List Content Control
 - o Date Picker Content Control
- Form Fields (Legacy)
 - o Text Form Field
 - o Check Box Form Field
 - o Drop-Down Form Field
- Document Properties

Field Types in Excel

- Cells

Can be named / unnamed. Merged cells are also supported. Formatting options for text, numbers, dates, etc., can be applied.
- Document Properties

Field Types in PDF

- Text Fields
- Choice Fields
- Button Fields

Supported Index Data Field Types

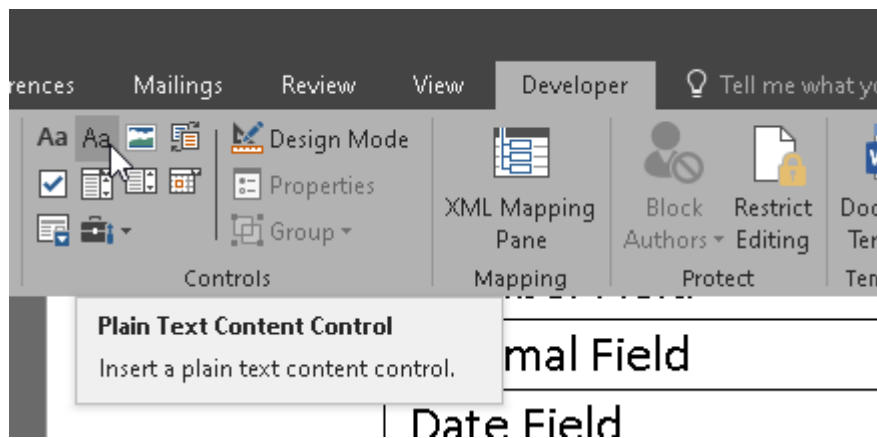
	Word	Excel	PDF
Text Field	yes	yes	yes
Number Field	yes**	yes	yes*
Decimal Field	yes**	yes	yes*
Date Field	yes	yes	yes*
Date and Time Field	yes	yes	yes*
Check Box (2-state)	yes	yes	yes
Table Field	yes	yes	no

* These fields will be treated as text. There are no formatting options

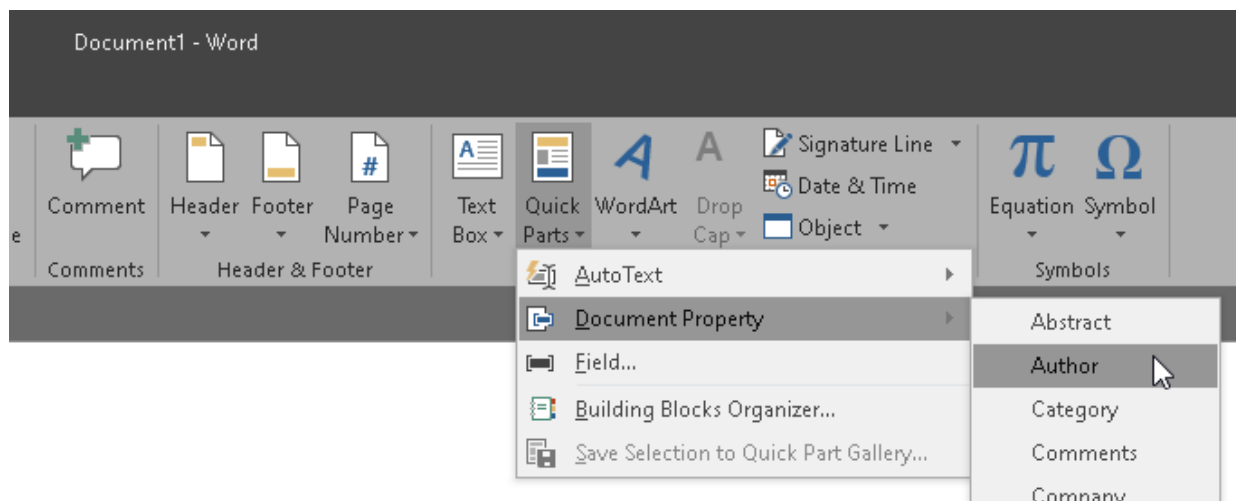
** These fields will be treated as text when using Content Controls

Inserting Content Controls or Form Fields into a Word Template

Content Controls or Form Fields can be inserted using the [Developer-Tab](#) in Word.



Using “Quick Parts” to insert Content Controls is currently not supported. Fields that are inserted in this manner store data in a certain XML format instead of plain text.



Map Index Data to Template Fields

Index data can be mapped to template fields by scripting. This is similar to the Update Index Data Task.

The document will have the same name as the template unless it is renamed by scripting:
`Profile.SetDocumentName("Document name");`

Mapping Table Columns

A Column from a Therefore™ Table Field can be linked directly to a Content Control or Form Field.

In this screenshot column ‘ItemNo’ is mapped to the Content Control ‘ItemNo’:

Add File to Document

Document Creation

Storage Options

Template: Therefore Invoice Template Del.dotx

Script:

Microsoft Office Control	Assignment
Content Controls	
BuyerName	IndexData.GetField("Customer_Name")
BuyerAddress	IndexData.GetField("Customer_Address")
BuyerCountry	IndexData.GetField("Customer_Country")
InvoiceNo	IndexData.GetField("Invoice_No")
InvoiceDate	IndexData.GetField("Invoice_Date")
TotalAmount	IndexData.GetField("Total")
ItemNo	IndexData.GetField("ItemNo")
ItemDesc	IndexData.GetField("ItemName")
ItemAmount	IndexData.GetField("ItemAmount")
ItemPrice	IndexData.GetField("ItemPrice")

Add Delete

OK Cancel

For Microsoft Excel templates, a cell-range needs to be specified in order to indicate to Therefore[™] that each value should be written into a separate cell. A range can either be named or specified. For example: "B4:B6".

Create New Document

Document Creation

Storage Options

Save Document

Document Creation

Template:

Basic Template.xlsx

Script:

Profile.SetDocumentName("Invoice" +

Microsoft Office Control	Assignment
Cells	
Field1	IndexData.GetField("Customer_Name")
Range1	IndexData.GetField("ItemName")
date3	IndexData.GetField("Invoice_Date")
B4:B6	IndexData.GetField("ItemNo")
Range3	[1005,2006,3007]
Sheet1!SheetField1	IndexData.GetField("Invoice_Amount_ne
Merged1	IndexData.GetField("Verified")
Merged2	IndexData.GetField("Customer_Number")
Merged3	IndexData.GetField("Customer_Address")
Font1	IndexData.GetField("Customer_Number")

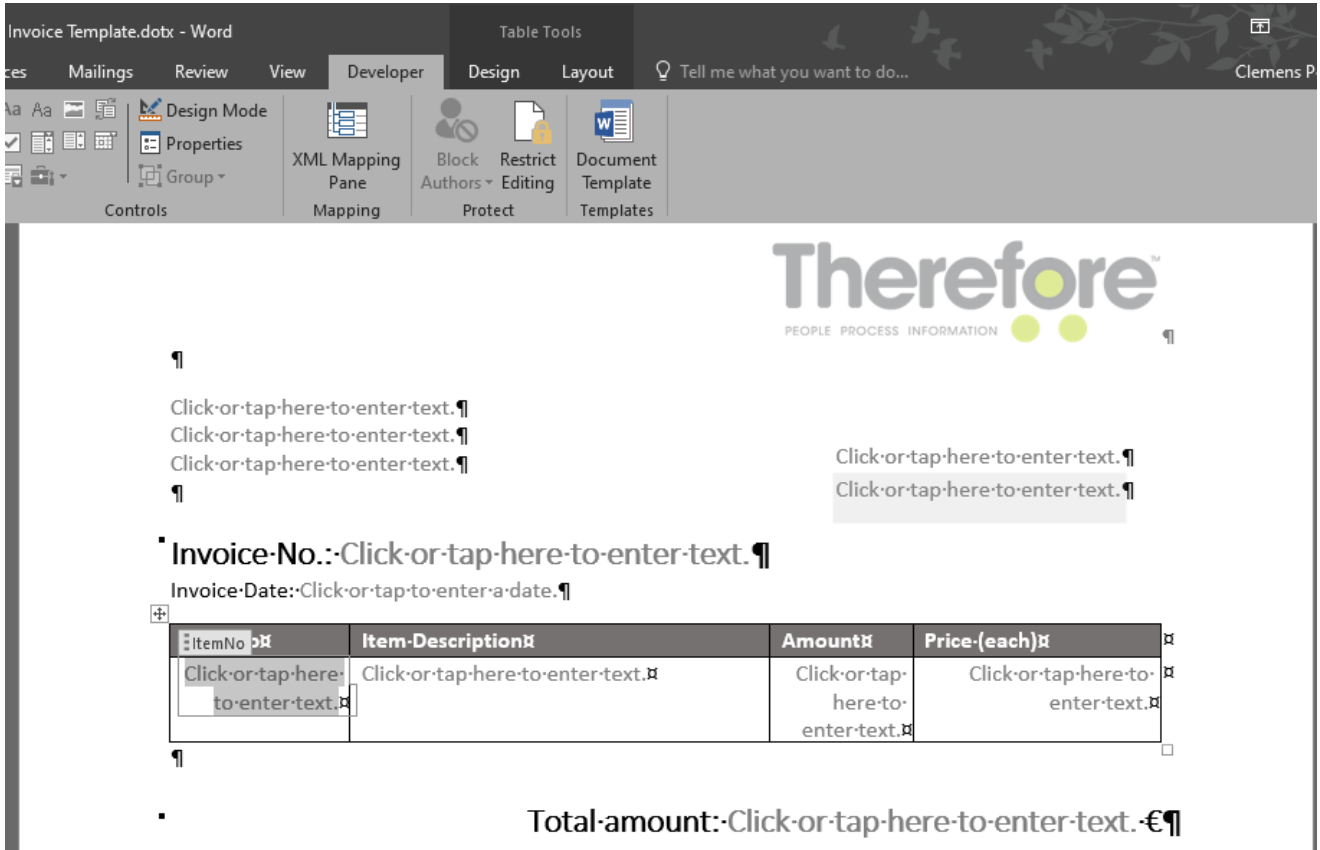
Add

Delete

OK

Cancel

For Microsoft Word templates, the Content Control / Form Field needs to be placed inside a table in order to indicate to Therefore™ that each value should be written into a separate table row.



Invoice Template.dotx - Word

Table Tools

Developer Design Layout Tell me what you want to do...

Properties XML Mapping Pane Mapping Block Restrict Authors Editing Protect Document Template Templates

Therefore
PEOPLE PROCESS INFORMATION

Click-or-tap-here-to-enter-text.

Click-or-tap-here-to-enter-text.

Click-or-tap-here-to-enter-text.

Click-or-tap-here-to-enter-text.

Click-or-tap-here-to-enter-text.

Invoice-No.: Click-or-tap-here-to-enter-text.

Invoice-Date: Click-or-tap-here-to-enter-a-date.

ItemNo	Item-Description	Amount	Price-(each)
Click-or-tap-here-to-enter-text.	Click-or-tap-here-to-enter-text.	Click-or-tap-here-to-enter-text.	Click-or-tap-here-to-enter-text.

Total-amount: Click-or-tap-here-to-enter-text.-€

If there are more values than empty rows Therefore™ will create a new empty row and copy the Content Control or Form Field.

As long as there are empty table cells available no new rows will be generated. If a non-empty table cell already exists below a new row is created in-between. This means that the existing values are shifted down.

Therefore™ will generate as many rows as required. Values are always inserted vertically - they will always be written into the next row, not into the next column.

If the Content Control or Form Field is not placed inside a table cell all array values will be inserted into that one field as a comma-separated string.

7.6 Building Loops with NextRequest

The documentation on the Therefore™ Connector with Microsoft Dynamics 365 Business Central shows an example of how to build loops using NextRequest when working with multiple REST calls in Therefore™.

Please find the documentation here:

https://therefore.net/help/Online/en-us/AR/conn_md365_bc_cpi.html

8. Integration with MS Office

8.1 Extract table index field into an Office template

In the event that the user intends to export all information from the table index to a Microsoft Office template (when a category contains a table index), the following instructions apply:

Date Ajout	Status	Champ Libre	Chef projet
14/10/2020	A Planifier	Ligne 1	Cédric Le Guilcher
14/10/2020	En attente de validation	Ligne2	Fabien Chaussade
14/10/2020	Livraison Partielle	Ligne 3	Fouzi Aissaoui

1. Begin by creating a simple Microsoft Word template, where all information is on the same line.

A Planifier, En attente de validation, Livraison Partielle	Cédric Le Guilcher, Fabien Chaussade, Fouzi Aissaoui	Ligne 1, Ligne2, Ligne 3

2. Then use this script (in the assignments) to implement the most effective way to get all information in Microsoft Word (or Excel) to be written line by line:

```
IndexData.GetField("TableText").join("\n")
```



Note:

This applies to all versions of Therefore™.

8.2 GetControl

Overview

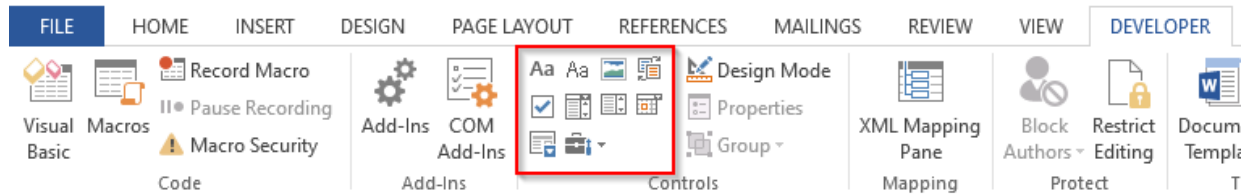
To retrieve index data from a Word document it is possible to use the following script functions.

- GetControl
- GetFormField

These can be used in an indexing profile for Microsoft Word.

GetControl

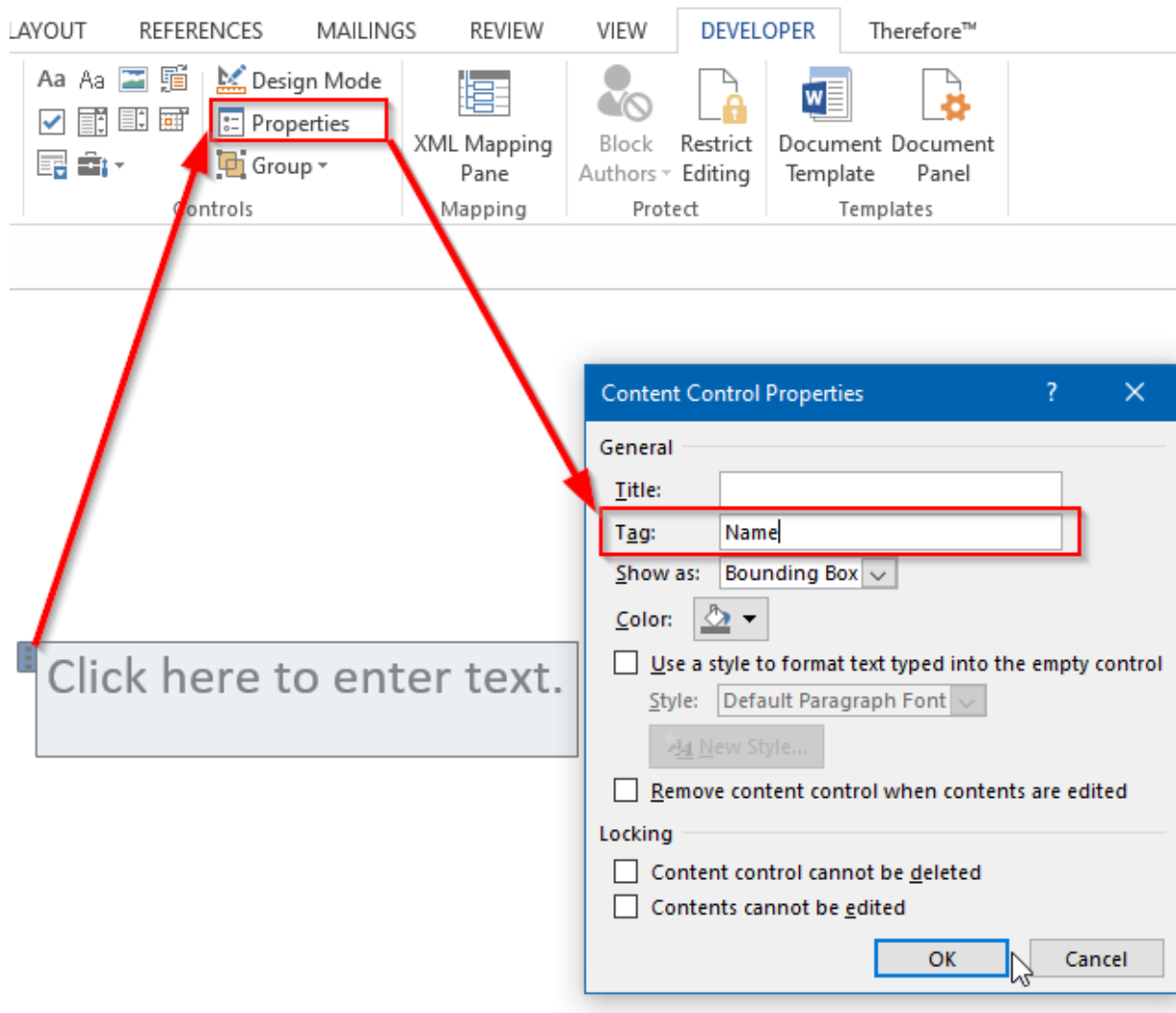
With GetControl the content of a content control can be retrieved. The available content controls are accessible in the Developer ribbon in Microsoft Word. The Developer ribbon is by default not visible and has to be activated manually in the Options.



The following content controls are supported:

- Rich Text Content Control
- Plain Text Content Control
- Check Box Content Control
- Combo Box Content Control
- Drop-Down List Content Control
- Date Picker Content Control

To be able to use the content control with the GetControl function it is necessary to give the content control a "tag".



This will be the name which can be used in the GetControl function.

```
var name = Word.GetControl("Name")
```

Return Values

Content Control	Return value type
Rich Text Content Control	string or undefined
Plain Text Content Control	string or undefined
Check Box Content Control	bool (true/false)
Combo Box Content Control	string or undefined
Drop-Down List Content Control	string or undefined
Date Picker Content Control	string or undefined

The return value will be undefined when a content control is empty or no item was chosen.

Combo Box and Drop-Down List

The Combo Box and Drop-Down List content control allow setting an internal value which should be used when the value is retrieved.

Display Name	Value
Invoice	Inv
Contract	CTR

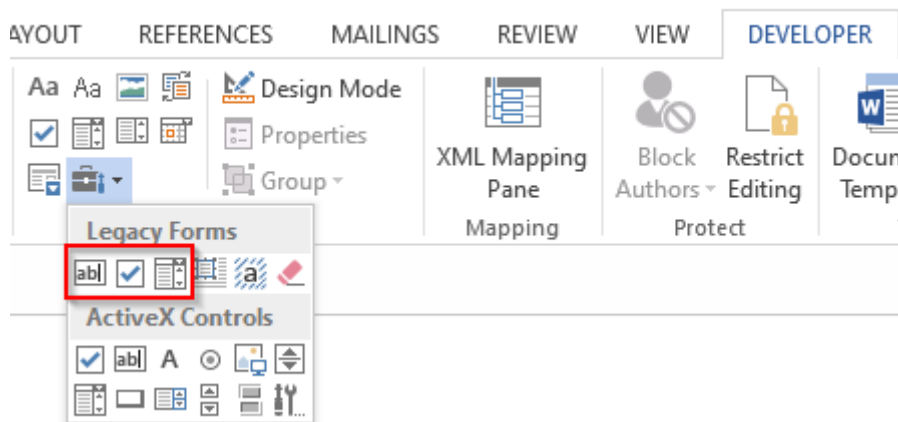
In the sample above, the user would see either "Invoice" or "Contract" in the Microsoft Word document. When this content control is retrieved via the GetControl function it will have either "Inv" or "CTR" as its value.

```
var type = Word.GetControl("Document Type")
// type is either "Inv" or "CTR"
```

If something is written into the combobox while no item is selected the value will be what is written into the combobox.

GetFormField

Form fields are the predecessor of content controls. They are available under 'Legacy Formats' as shown below:



The following Form fields are supported:

- Text Form Field
- Check Box Form Field
- Drop-Down Form Field

Similar as for content controls the form field needs to be given a name. Enter the name into the 'Bookmark' field:

```
var name = Word.GetFormField("Name")
```

Return Values

The return value will always be a string even for an empty field.

Form Field	Return Value type
Text Form Field	string
Check Box Form Field	string ("0" for unchecked; "1" for checked)
Drop-Down Form Field	string

ActiveX Controls

ActiveX controls are not supported.

Related

See also [GetProperty](#).

8.3 GetFormField

See [GetControl](#).

8.4 GetCell

Overview

To retrieve data from an Excel sheet use one of the following script functions:

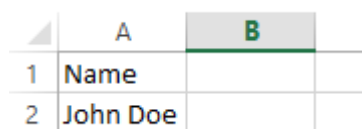
- GetCell
- GetCells

The GetCell function returns the value of a single cell while the GetCells function returns an array of multiple cells.

Usage Samples

Single Cell

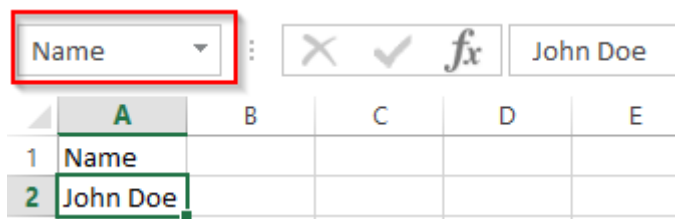
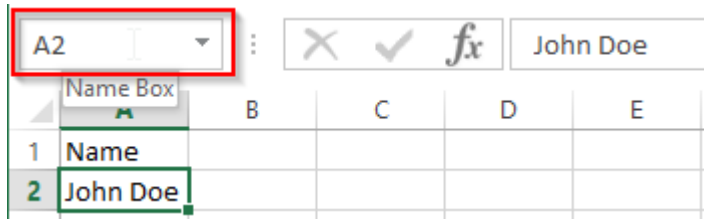
To identify a cell use the normal Excel notation, for example "A1".



	A	B
1	Name	
2	John Doe	

```
var name = Excel.GetCell("A2"); // will return "John Doe"
```

Alternatively, use an Excel Named Range. One way to give a cell a name is to simply type a new name for the cell into the "Name Box".



This name will also be shown as an available field in the indexing profile. This name can be used in the GetCell function.

```
var name = Excel.GetCell("Name");
```

Multiple Cells

Use the GetCells function to retrieve multiple cells. It is especially useful to fill index data tables. Similarly to GetCell, the Excel notation can be used for a range. GetCells will return an array of the cell range:

	A	B
1	Invoices	
2	Inv123	
3	Inv456	
4	Inv789	

```
var invoices = Excel.GetCells("A2:A4"); // from A2 to A4
```

It is also possible to define an Excel Named Range for multiple cells. Simply drag and select the cells and use the "Name Box" again.

Data from different sheets

By default, when just "A1" is used the data is retrieved from the currently active sheet. To access any sheet regardless of the currently active one the sheet name must be specified as well.

	A	B	C
1	Invoices		
2	Inv123		
3	Inv456		
4	Inv789		
		Sheet1	Sheet2

	A	B	C
1	Name		
2	John Doe		
3			
4			
		Sheet1	Sheet2

```
var invoices = Excel.GetCells("Sheet1!A2:A4");
```

```
var name = Excel.GetCell("Sheet2!A2");
```

Return value types

The return value types depend on the content of the cell. Cells with text content return a string, Cells with number content return numbers, etc.

Related

See also [GetProperty](#).

8.5 GetCells

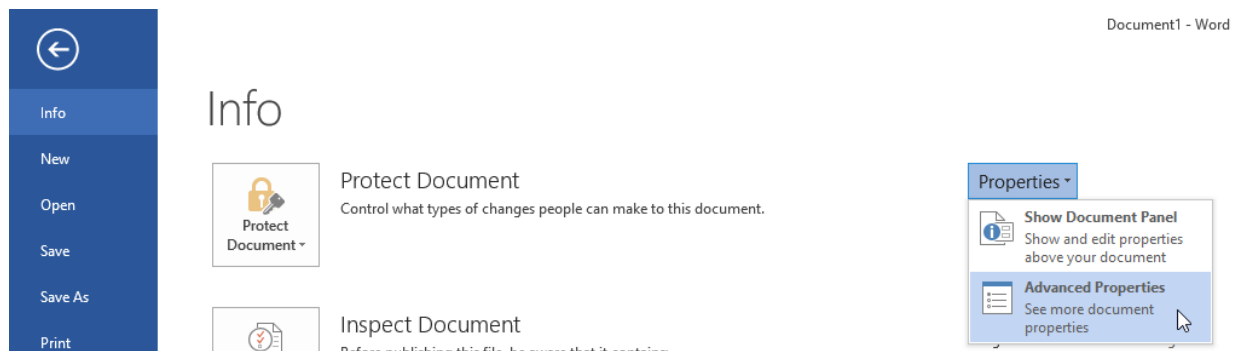
See [GetCell](#).

8.6 GetProperty

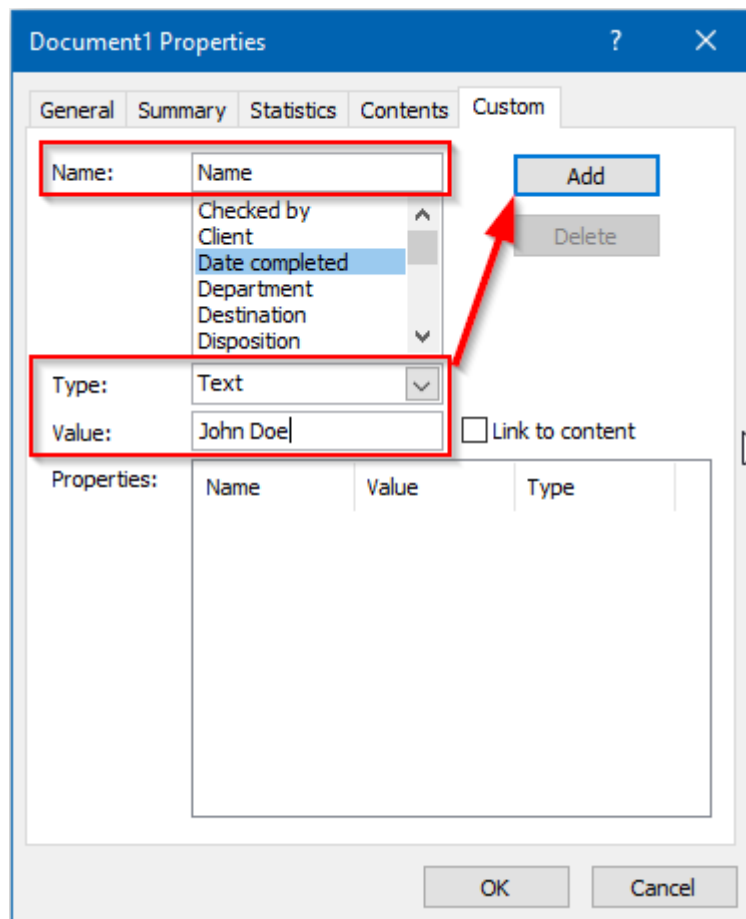
Overview

With `GetProperty` custom document properties can be retrieved. These are available in most Microsoft Office applications.

The menu can be found here:



Document properties can be added under the "Custom" - Tab.



Document1 Properties

General Summary Statistics Contents Custom

Name:

Checked by
Client
Date completed
Department
Destination
Disposition

Type: Text

Value: ☐ Link to content

Name	Value	Type
Name	John Doe	Text

OK Cancel

```
var name = Word.GetProperty("Name")
```

Legal

All rights reserved. No parts of this work may be reproduced in any form or by any means - graphic, electronic, or mechanical, including photocopying, recording, taping, or information storage and retrieval systems - without the written permission of the publisher.

"JavaScript" is a trademark of Oracle Corporation in the United States.

Any other 3rd Party Products that are referred to in this document, are the property of, and may be either trademarks and/or registered trademarks of the respective owners in the USA and/or other countries. The publisher and the author make no claim to these trademarks.

While care has been taken in the preparation of this document, the publisher and the author assume no responsibility for errors or omissions, or for damages resulting from the use of information contained in this document.

VERSION: 2023 - 1